



《模式识别》

第十章 卷积神经网络及经典框架

马锦华

<https://cse.sysu.edu.cn/teacher/MaJinhua>

SUN YAT-SEN University



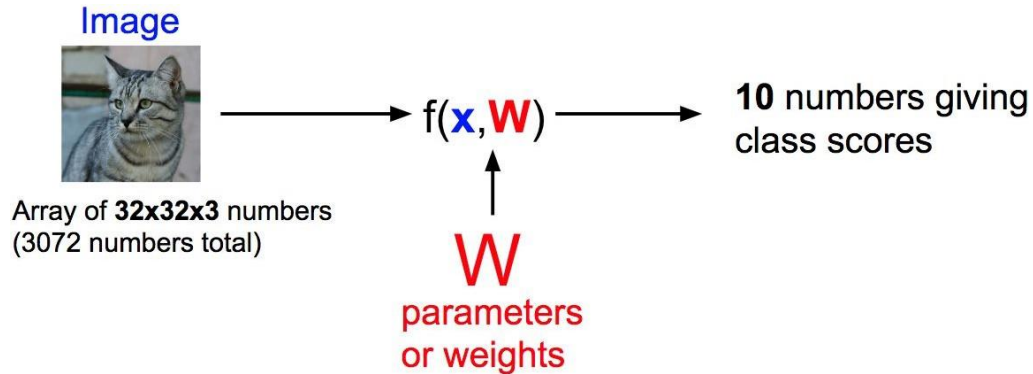
声明：该PPT只供非商业使用，也不可视为任何出版物。由于历史原因，许多图片尚没有标注出处，如果你知道图片的出处，欢迎告诉我们 majh8@mail.sysu.edu.cn.



课程目录（暂定）

❑	第一章	课程简介与预备知识	6学时
❑	第二章	特征提取与表示	6学时
❑	第三章	主成分分析	3学时
❑	第四章	归一化、判别分析、人脸识别	3学时
❑	第五章	EM算法与聚类	3学时
❑	第六章	贝叶斯决策理论	3学时
❑	第七章	线性分类器与感知机	3学时
❑	第八章	支持向量机	3学时
❑	第九章	神经网络、正则项和优化方法	3学时
❑	第十章	卷积神经网络及经典框架	6学时
❑	第十一章	循环神经网络	2学时
❑	第十二章	Transformer	2学时
❑	第十三章	自监督与半监督学习	2学时
❑	第十四章	开放世界模式识别	2学时

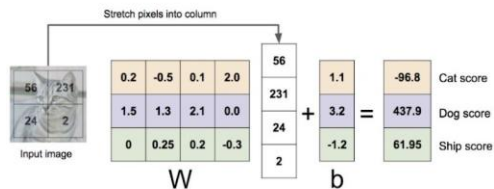
Recap: Image Classification with Linear Classifier



$$f(\mathbf{x}, \mathbf{W}) = \mathbf{W}\mathbf{x} + b$$

Algebraic Viewpoint

$$f(\mathbf{x}, \mathbf{W}) = \mathbf{W}\mathbf{x}$$



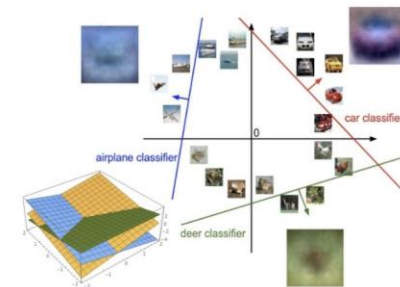
Visual Viewpoint

One template
per class



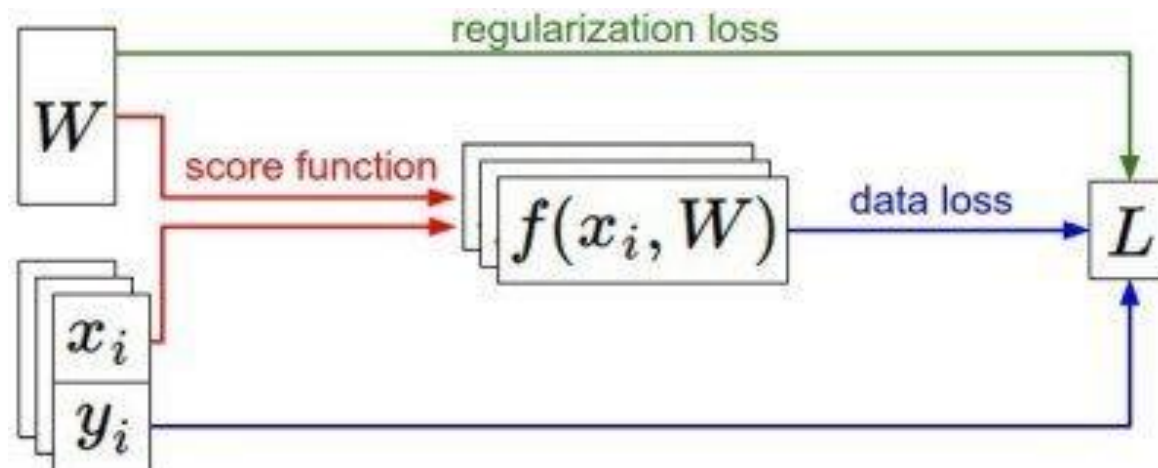
Geometric Viewpoint

Hyperplanes
cutting up space

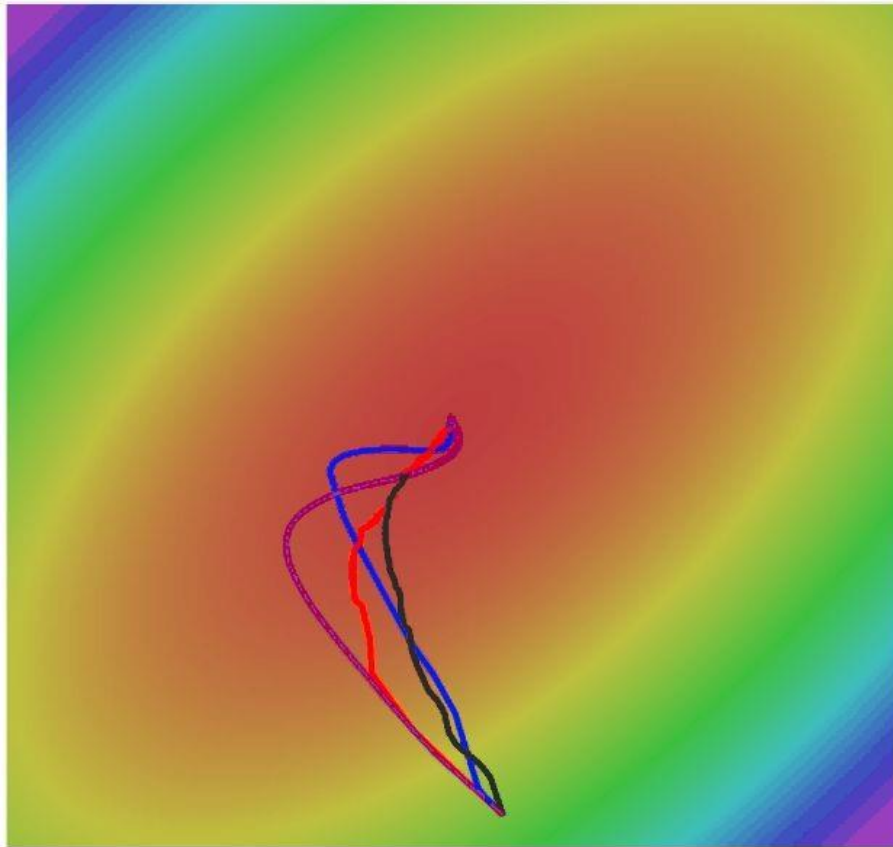


Recap: Loss Function

- We have some dataset of (x, y)
- We have a **score function**: $s = f(x; W) = Wx$
- We have a **loss function**:
- **Softmax**: $L_i = -\log\left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}}\right)$
- **SVM**: $L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$
- **Full loss**: $L = \frac{1}{N} \sum_{i=1}^N L_i + R(W)$



Recap: Optimization



-  *SGD*
-  *SGD+Momentum*
-  *RMSProp*
-  *Adam*

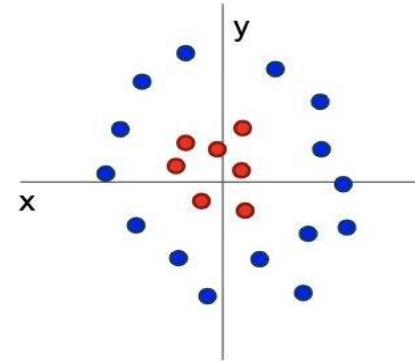
Problem: Linear Classifiers are not very powerful

Visual Viewpoint



*Linear classifiers learn
one template per class*

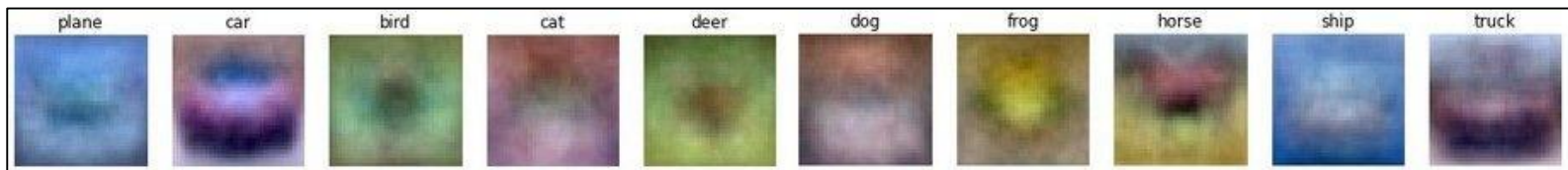
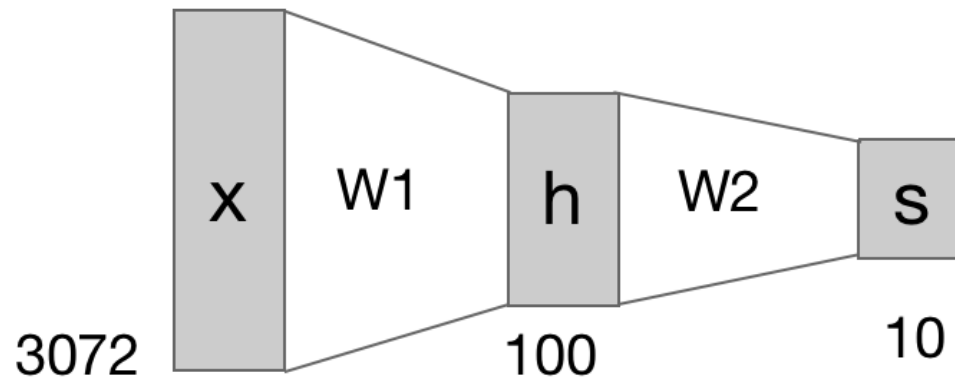
Geometric Viewpoint



*Linear classifiers
can only draw linear
decision boundaries*

Last time: Neural Networks

- Linear score function: $f = Wx$
- 2-layer Neural Network: $f = W_2 \max(0, W_1 x)$



Last time: Backpropagation

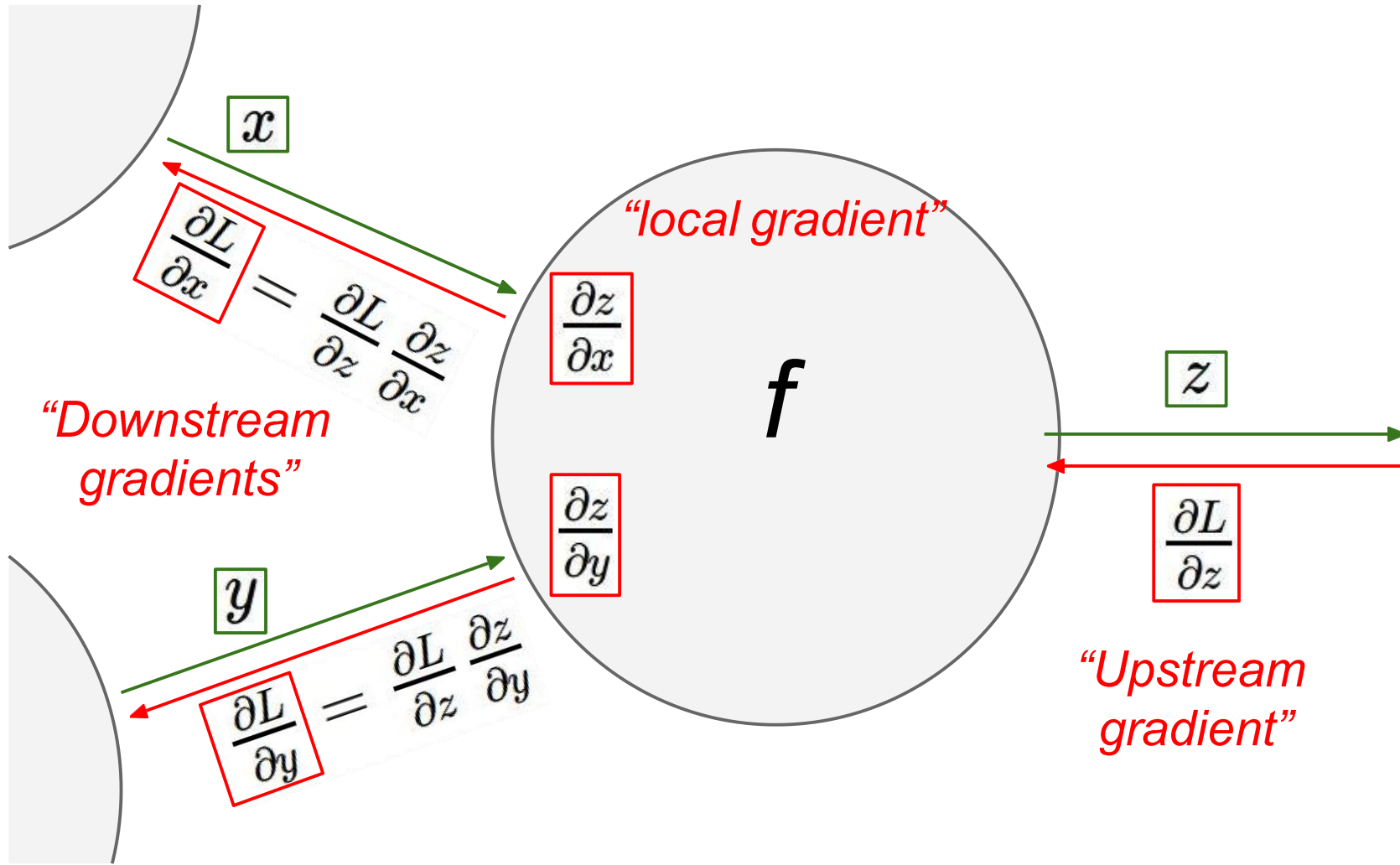


Image Classification: A core task in Computer Vision



*This image by Nikita is
licensed under CC-BY 2.0*

*(assume given a set of labels)
{dog, cat, truck, plane, ...}*



cat
dog
bird
deer
truck

Pixel space

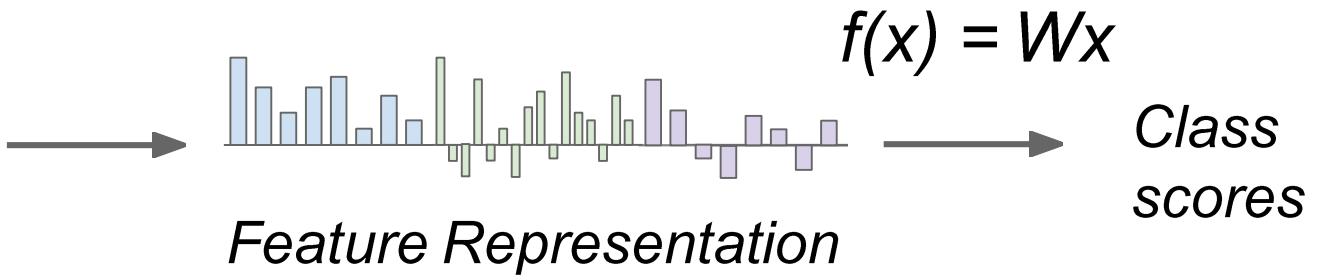


*Class
scores*

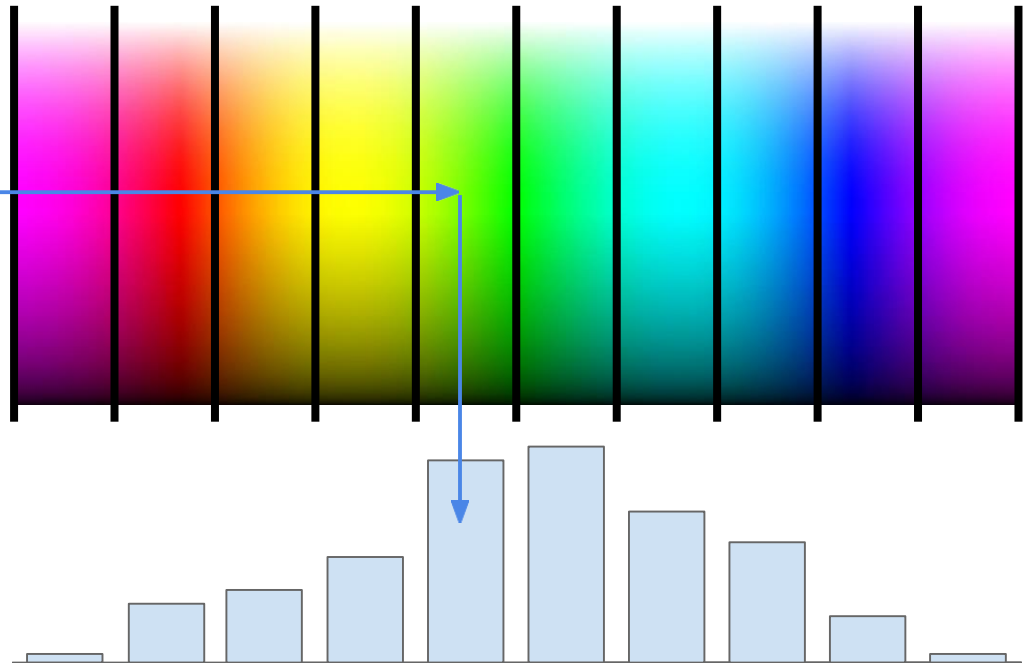
$$f(x) = Wx$$



Image features



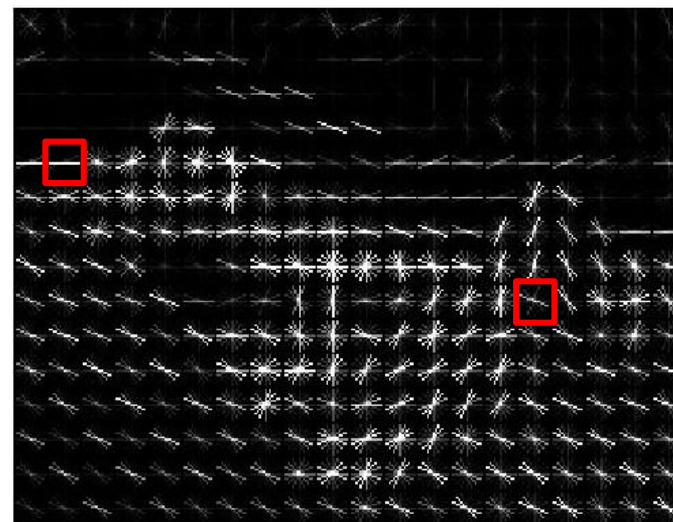
Example: Color Histogram



Example: Histogram of Oriented Gradients (HoG)



*Divide image into 8x8 pixel regions
Within each region
quantize edge direction
into 9 bins*



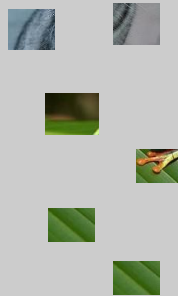
*Example: 320x240 image gets
divided into 40x30 bins; in
each bin there are 9 numbers
so feature vector has $30 \times 40 \times 9$
= 10,800 numbers*

Example: Bag of Words

Step 1: Build codebook



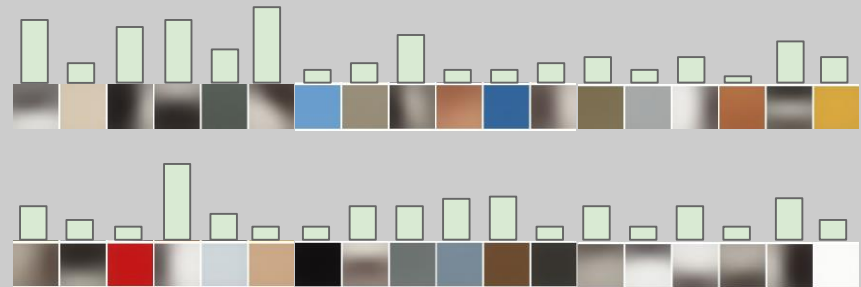
Extract random patches



Cluster patches to form "codebook" of "visual words"



Step 2: Encode images



Fei-Fei and Perona, "A bayesian hierarchical model for learning natural scene categories", CVPR 2005

Image Features

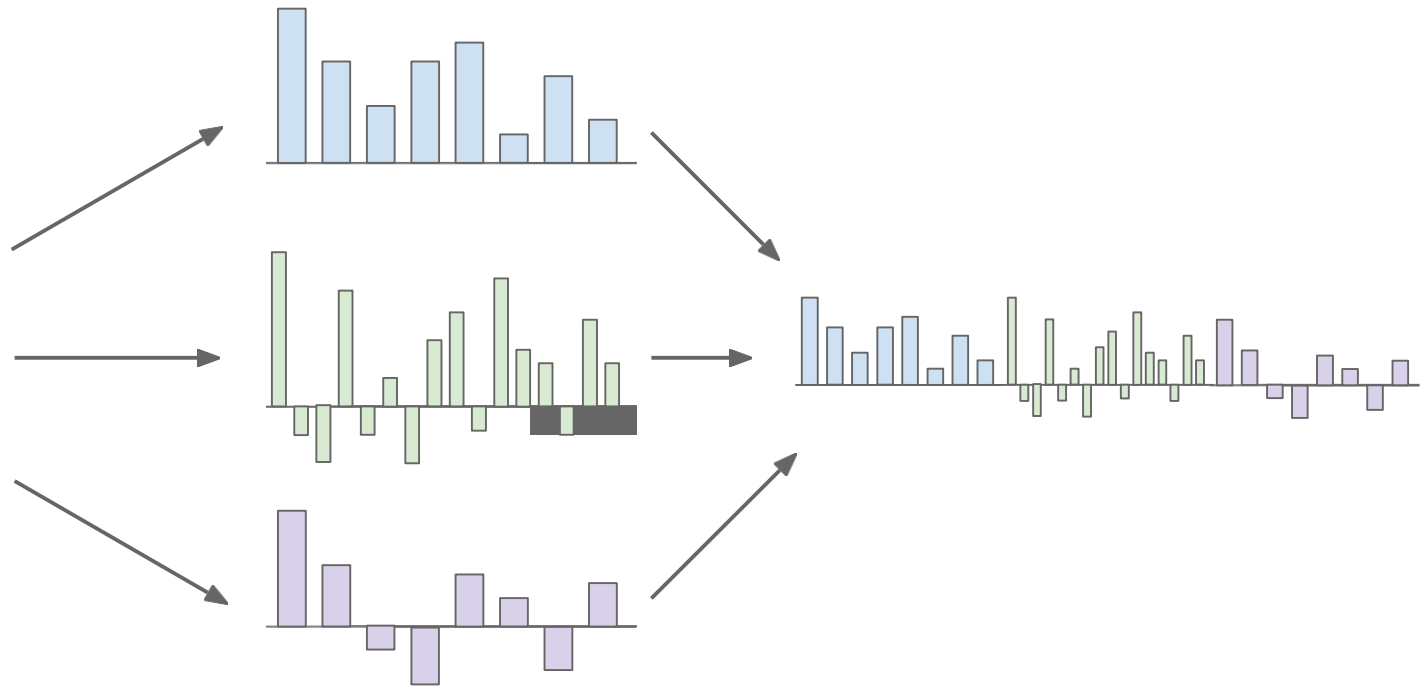
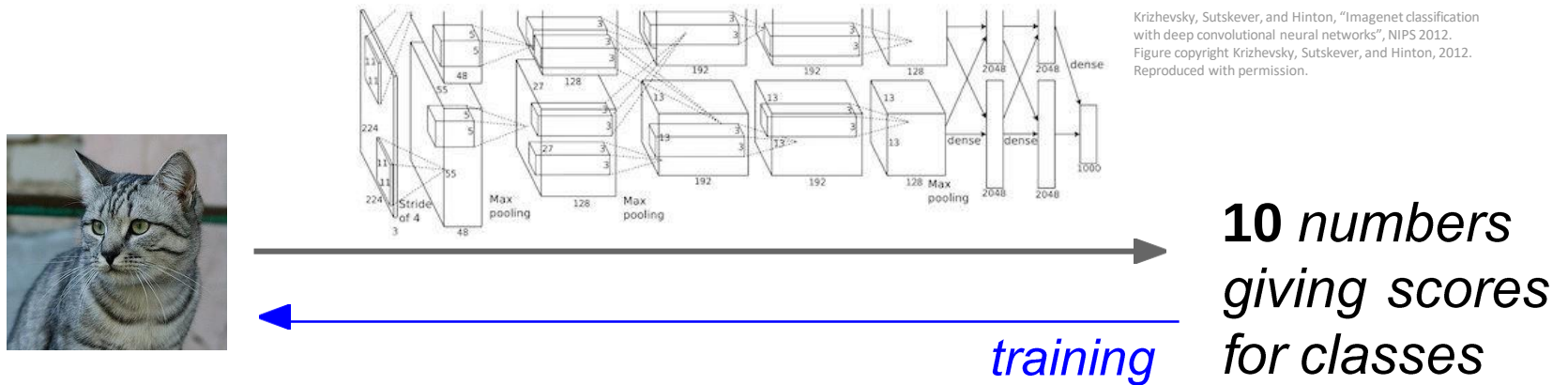
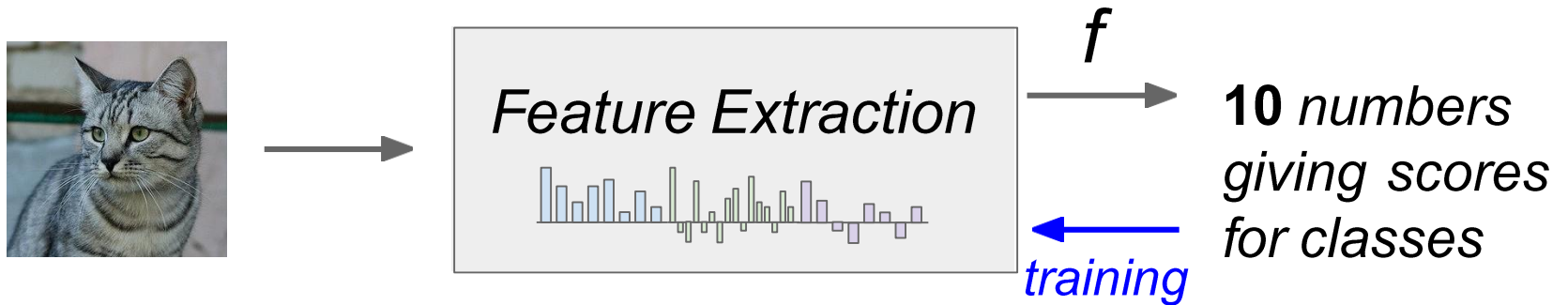


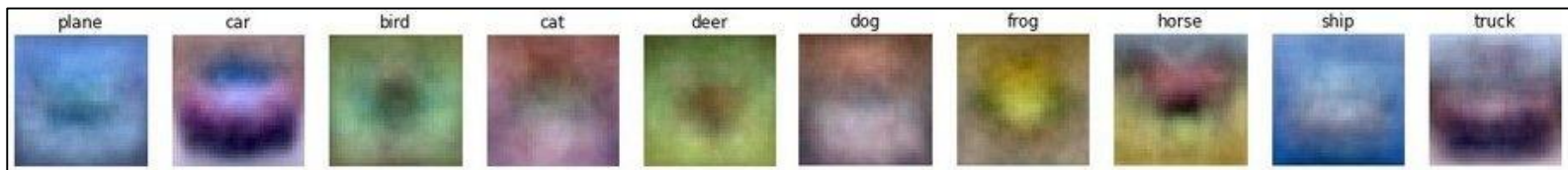
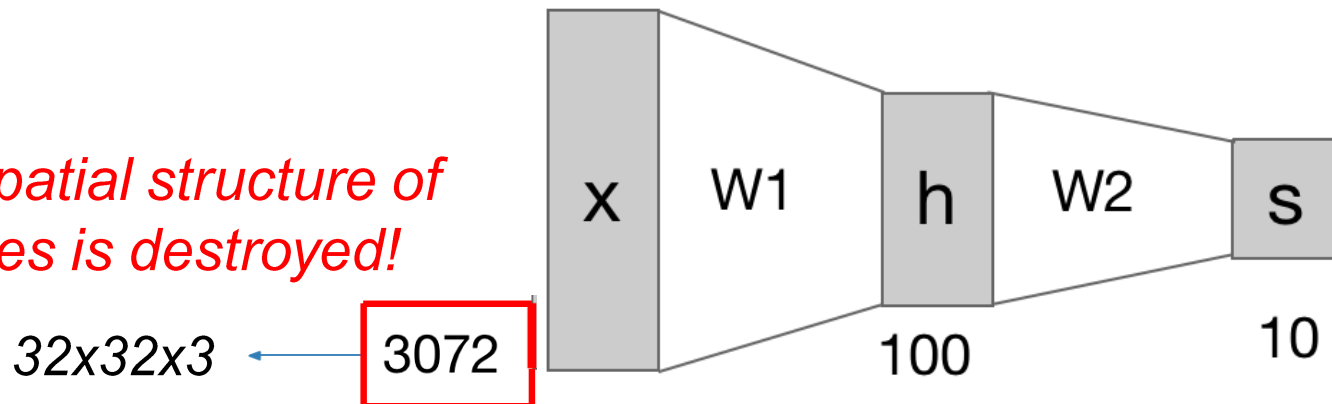
Image features vs. ConvNets



Last time: Neural Networks

- Linear score function: $f = Wx$
- 2-layer Neural Network: $f = W_2 \max(0, W_1 x)$

The spatial structure of images is destroyed!



Next: Convolutional Neural Networks

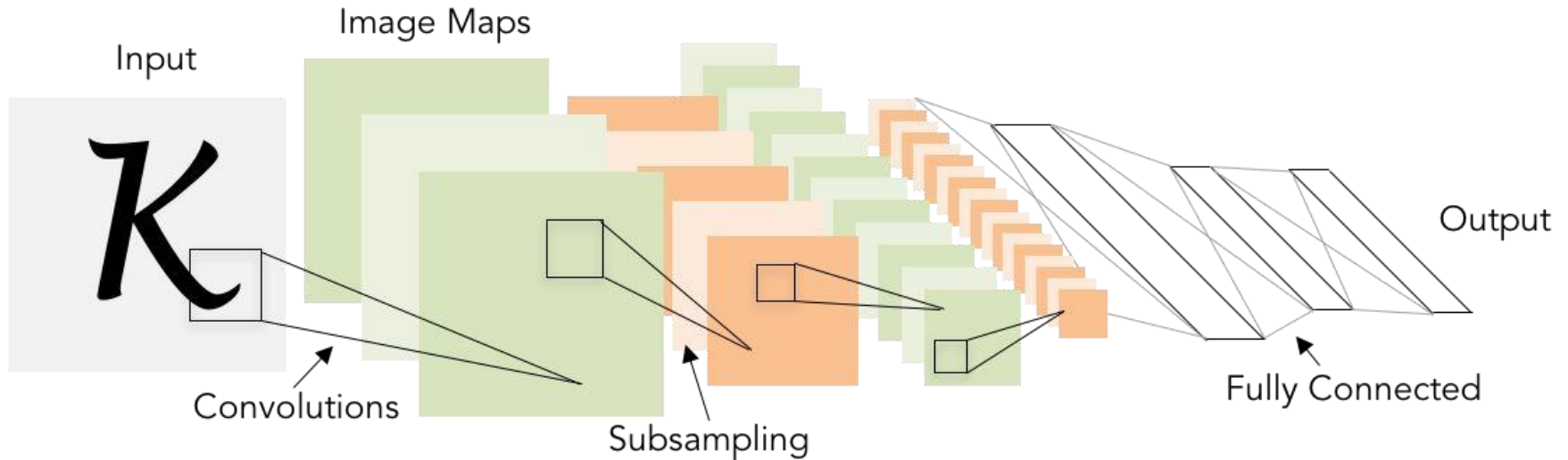
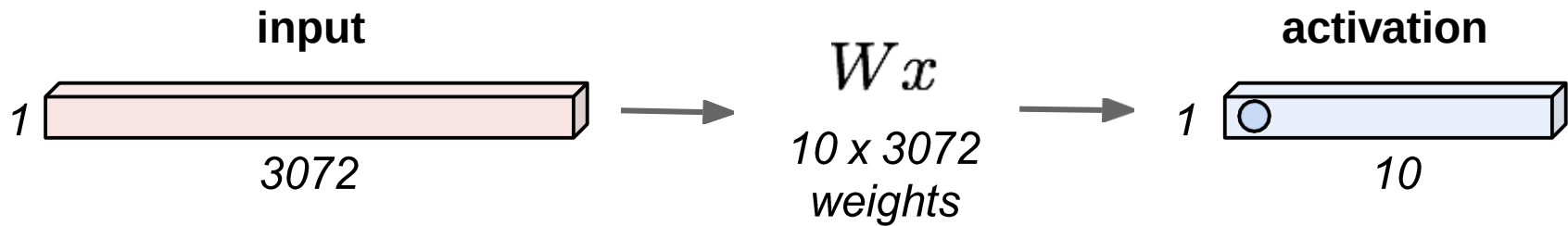


Illustration of LeCun et al. 1998 from CS231n 2017 Lecture 1

Convolutional Neural Networks

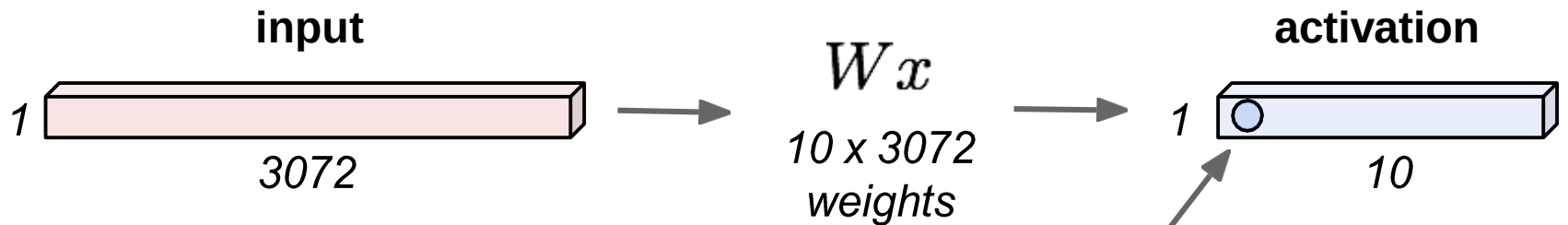
Recap: Fully Connected Layer

32x32x3 image -> stretch to 3072 x 1



Fully Connected Layer

32x32x3 image -> stretch to 3072 x 1



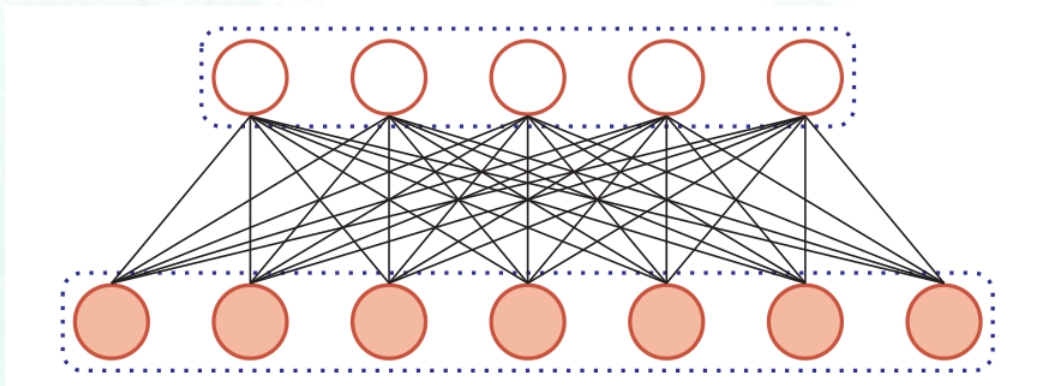
1 number:

the result of taking a dot product between a row of W and the input (a 3072-dimensional dot product)



全连接神经网络的问题

- 权重矩阵的参数非常多



- 很难提取局部不变性特征

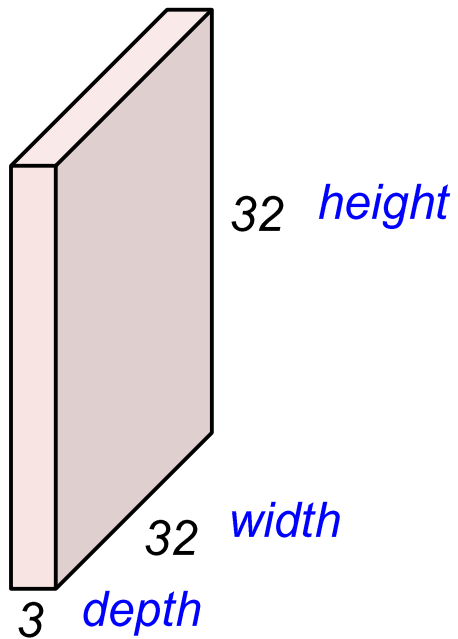
- 自然图像中的物体都具有局部不变性特征

- 尺度缩放、平移、旋转等操作不影响其语义信息



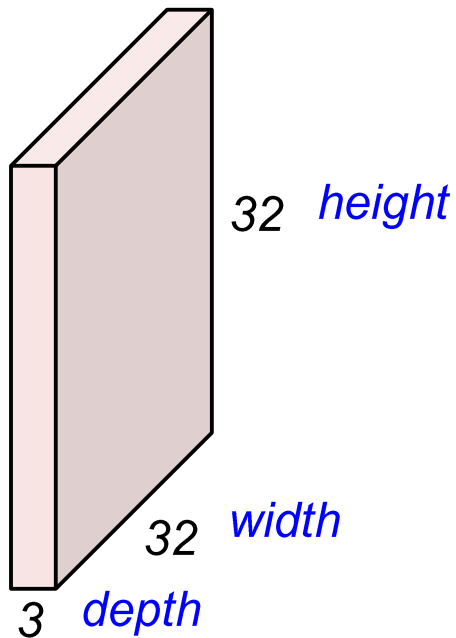
Convolution Layer

32x32x3 image -> preserve spatial structure

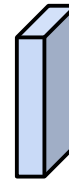


Convolution Layer

$32 \times 32 \times 3$ image



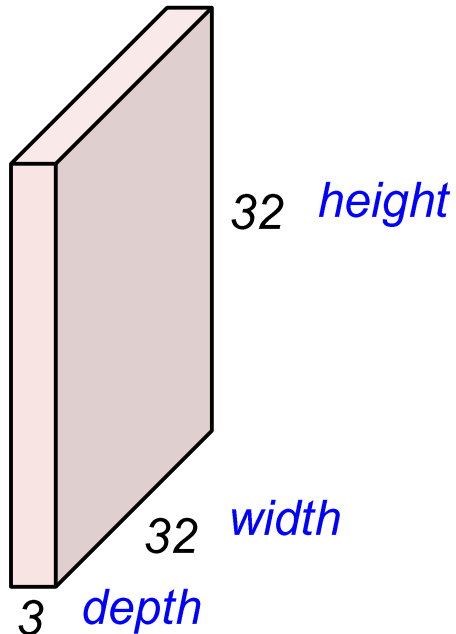
$5 \times 5 \times 3$ filter



Convolve the filter with the image
i.e. “slide over the image spatially,
computing dot products”

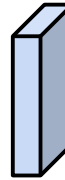
Convolution Layer

32x32x3 image



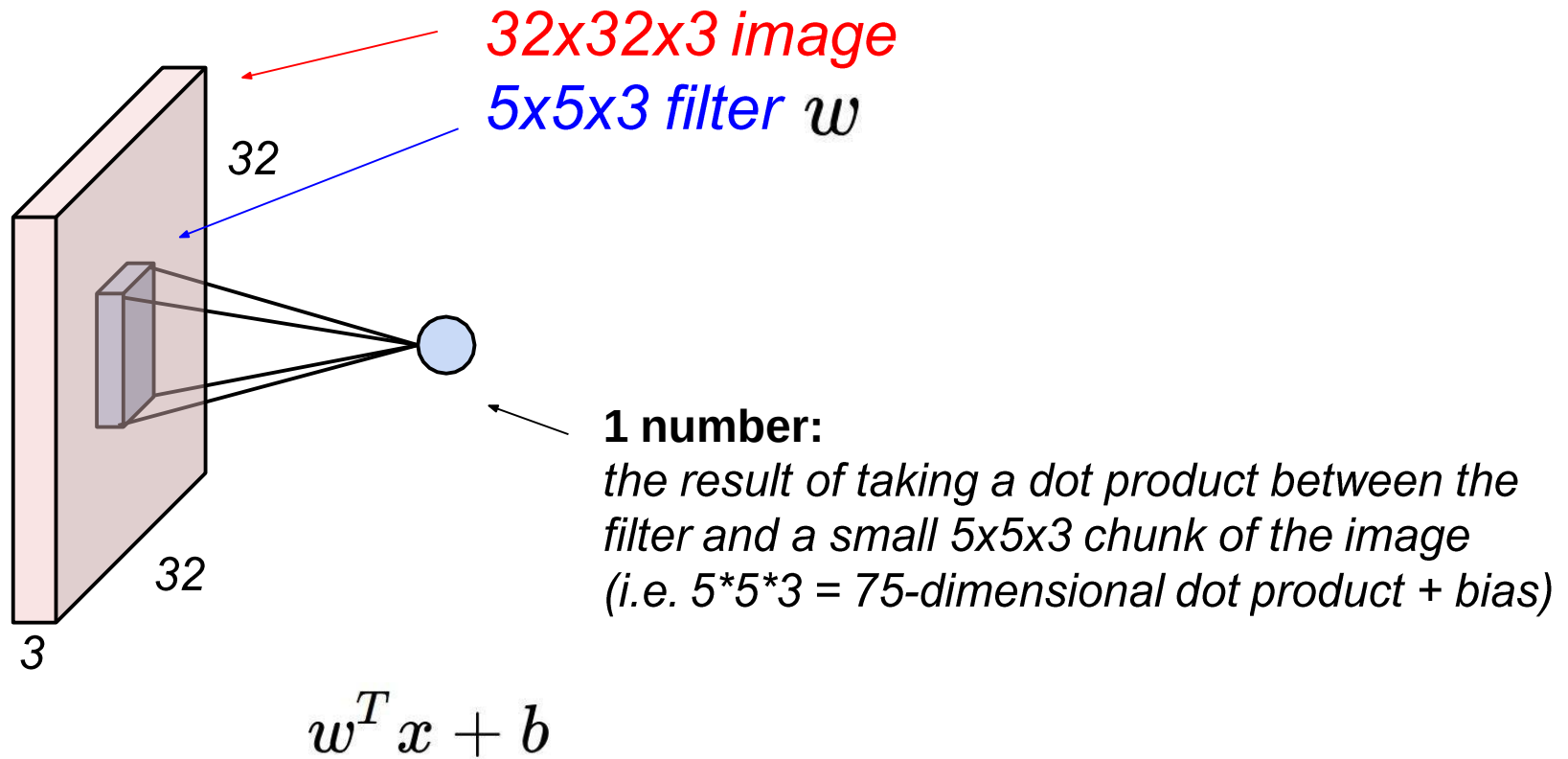
Filters always extend the full depth of the input volume

5x5x3 filter

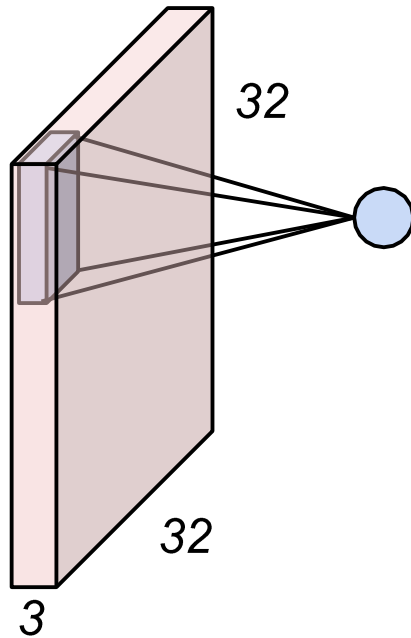


Convolve the filter with the image
i.e. “slide over the image spatially,
computing dot products”

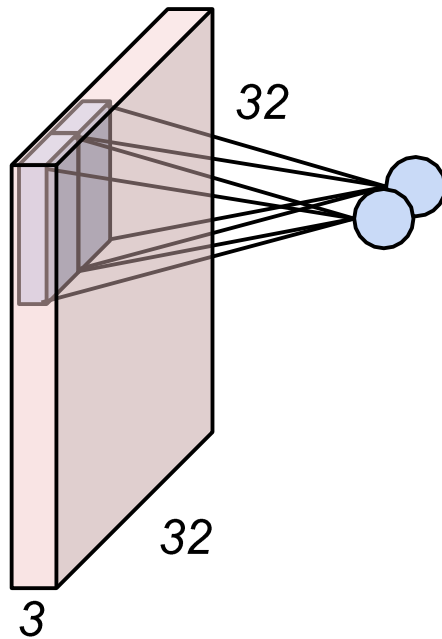
Convolution Layer



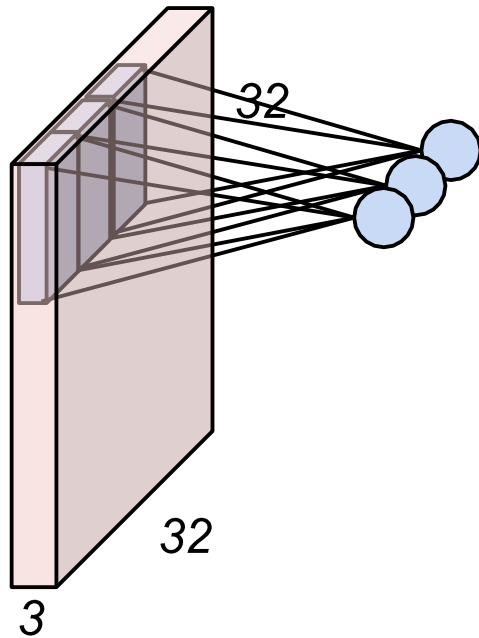
Convolution Layer



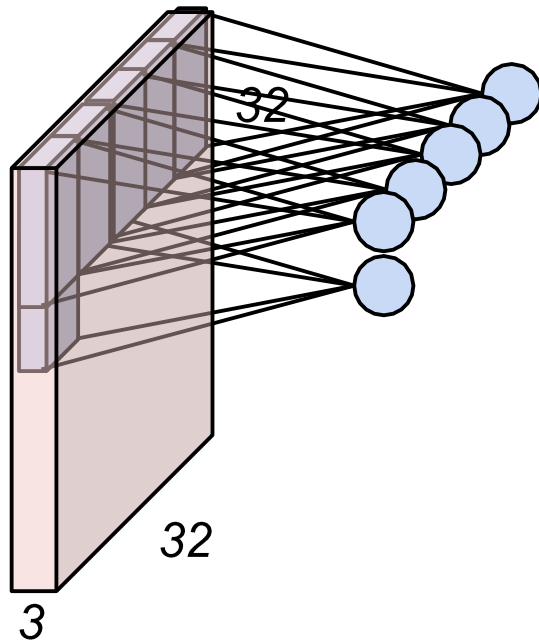
Convolution Layer



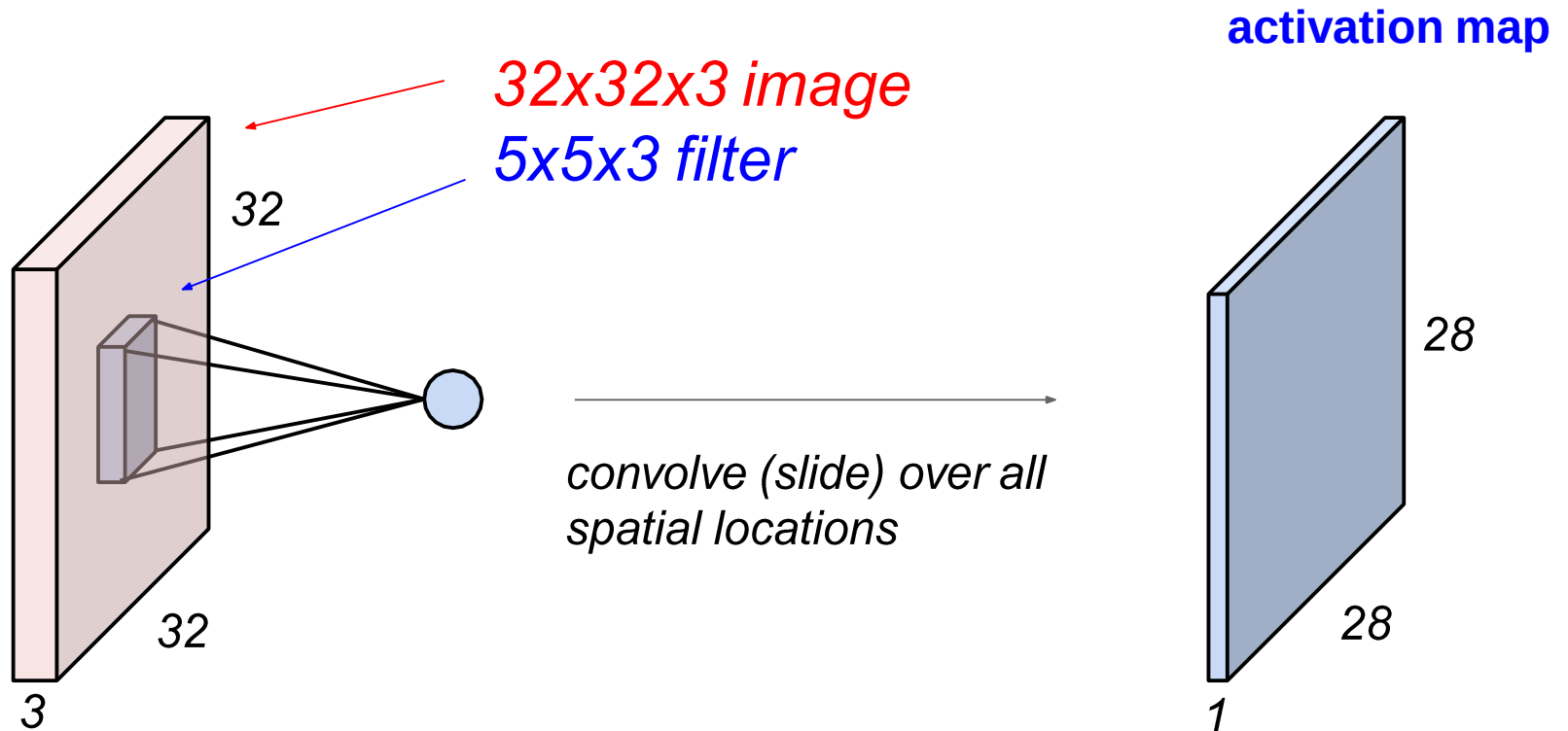
Convolution Layer



Convolution Layer

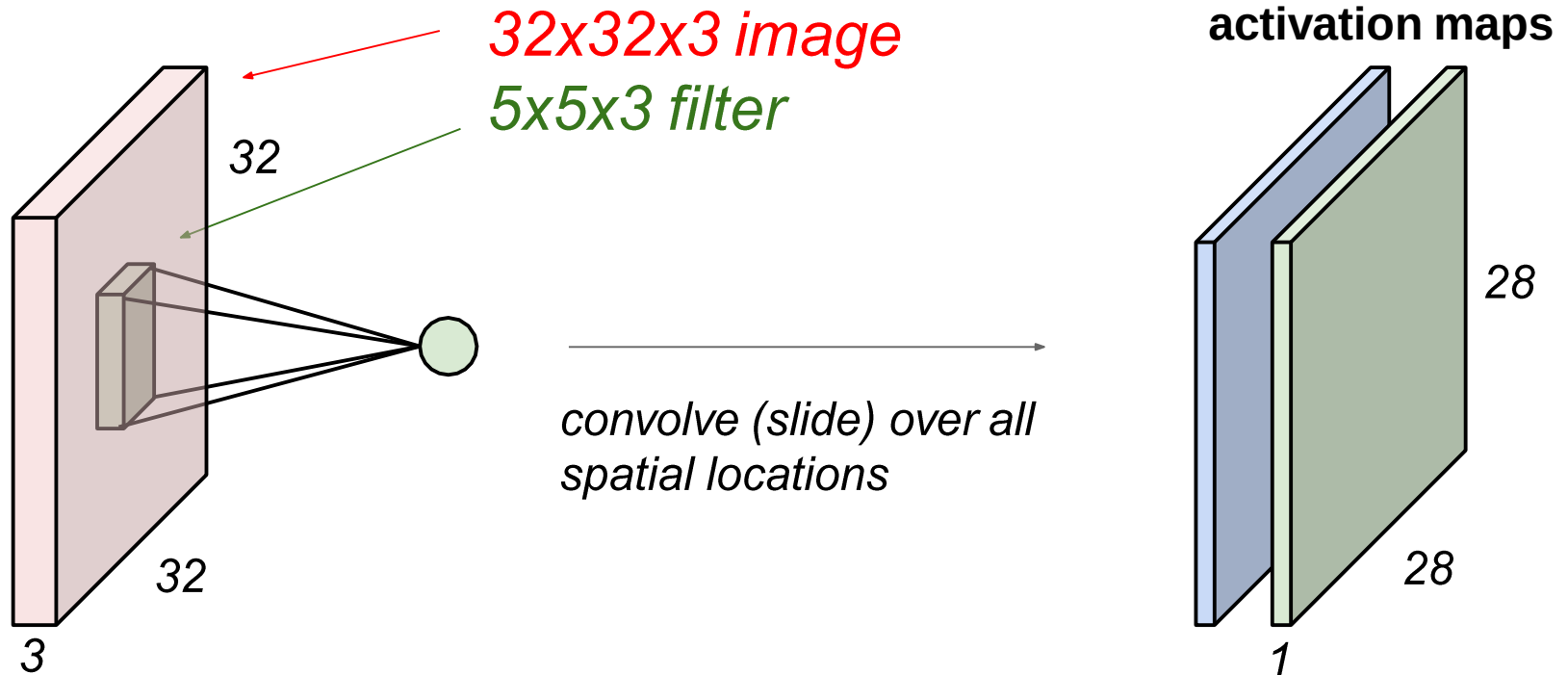


Convolution Layer

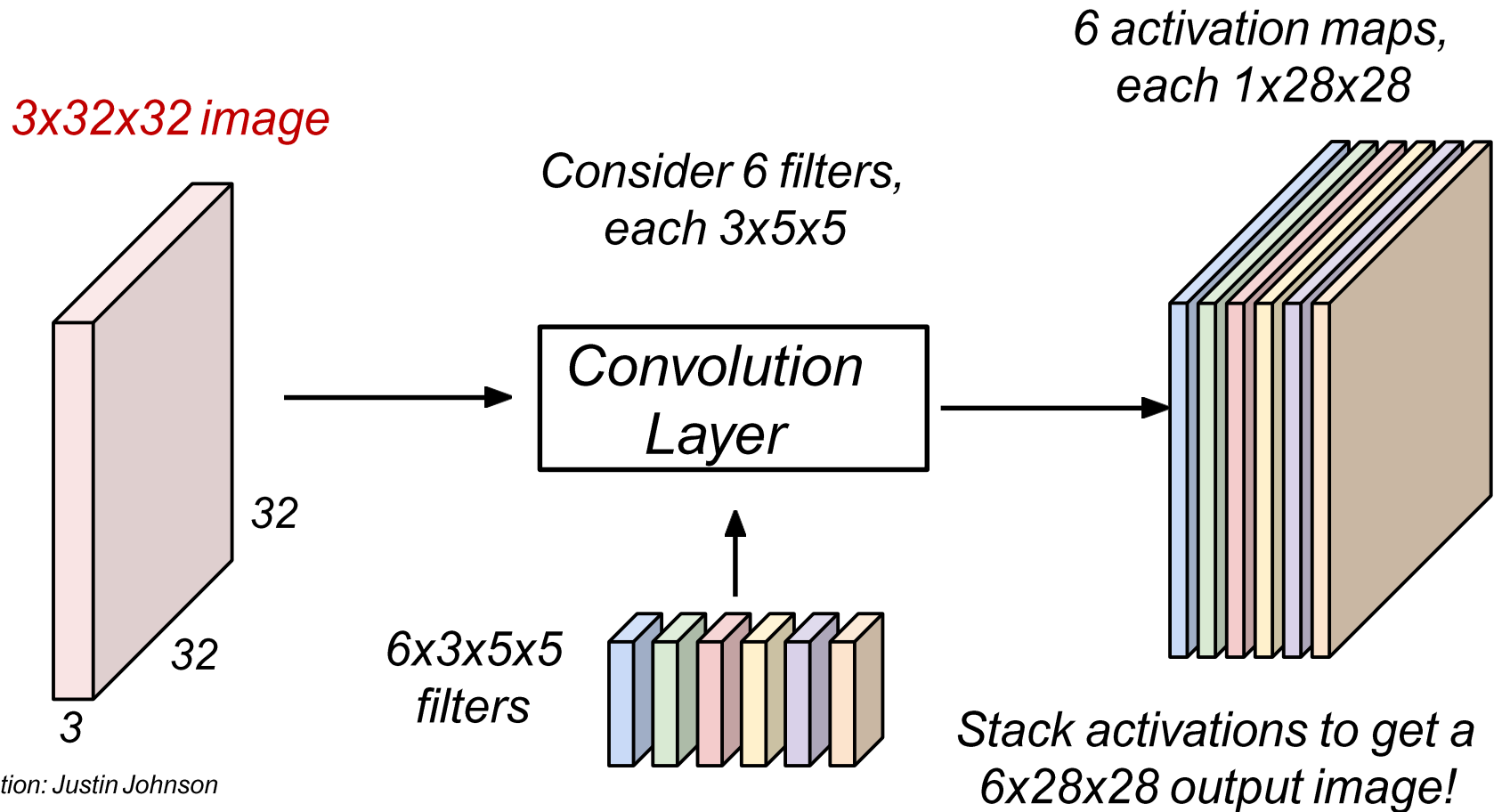


Convolution Layer

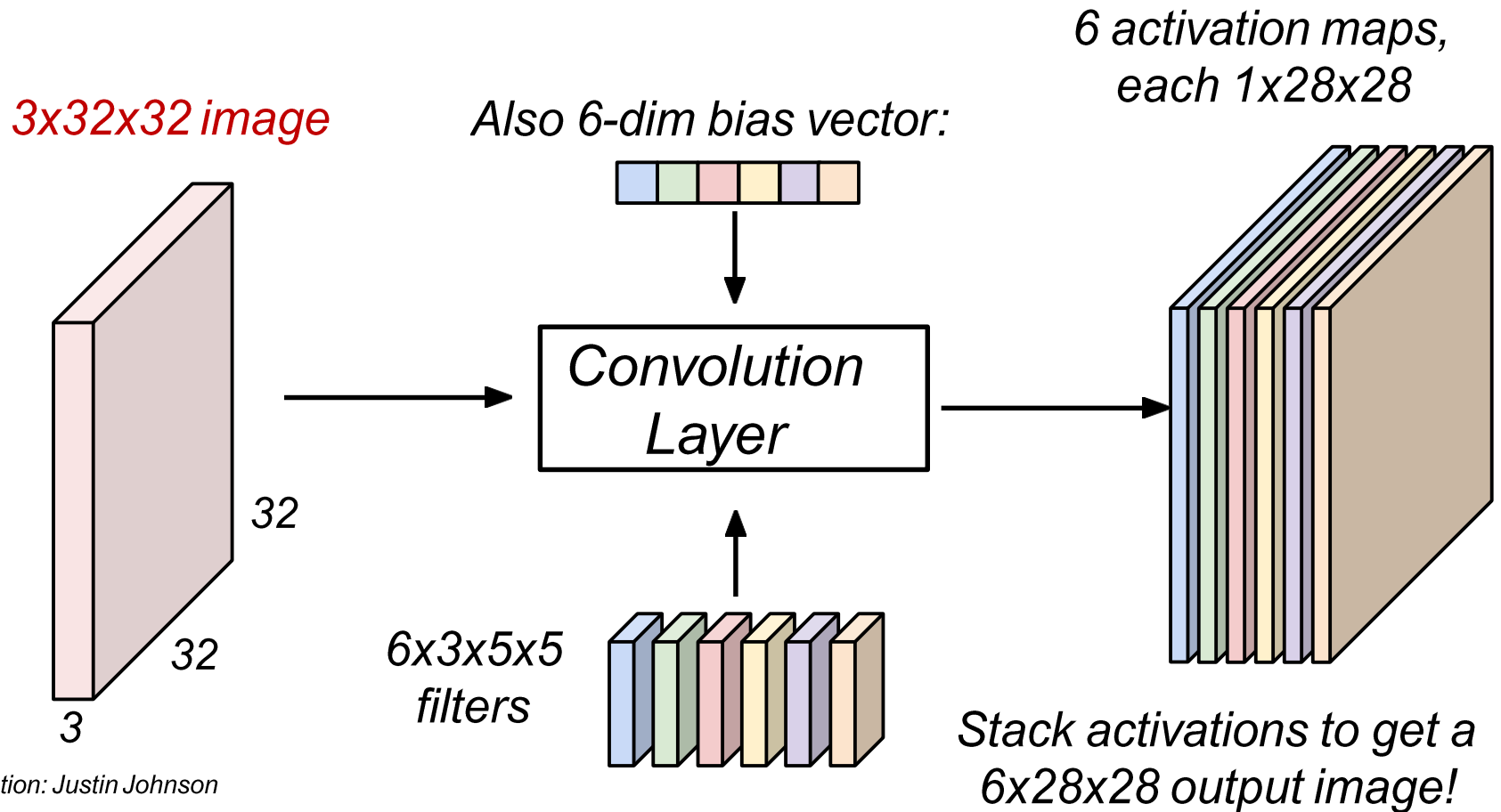
consider a second, **green** filter



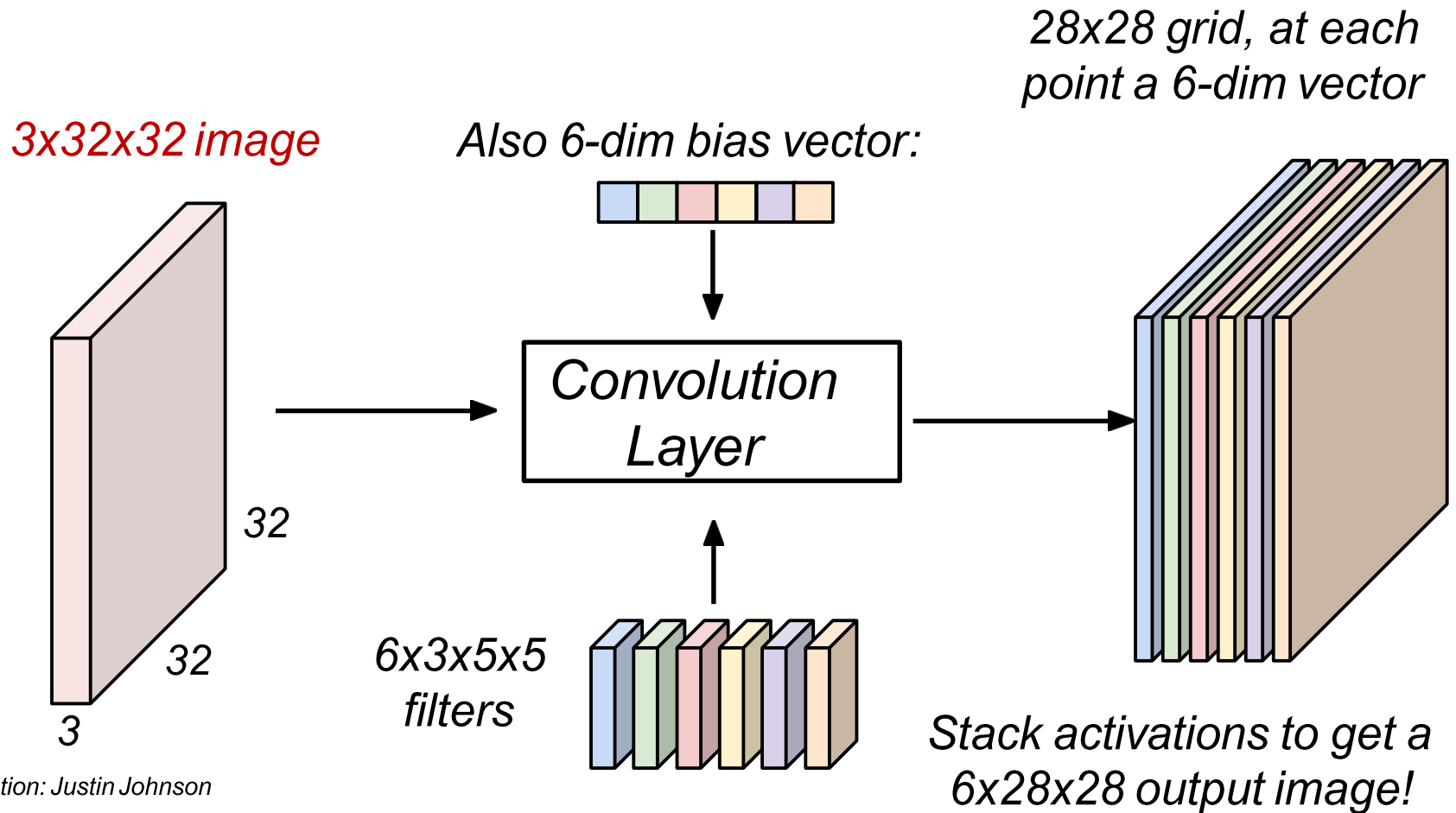
Convolution Layer



Convolution Layer



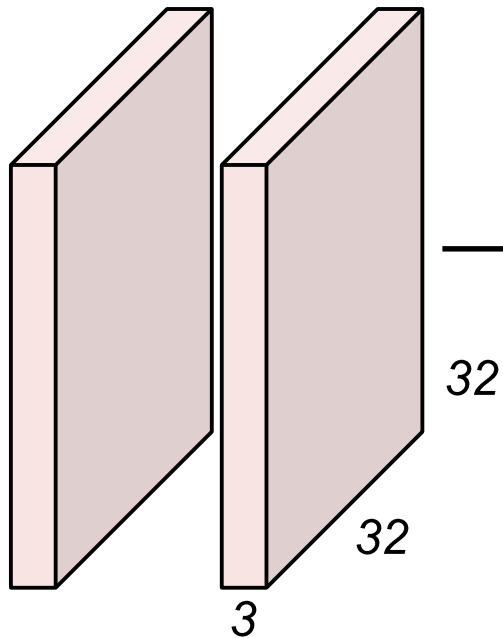
Convolution Layer



Slide inspiration: Justin Johnson

Convolution Layer

2x3x32x32
Batch of images

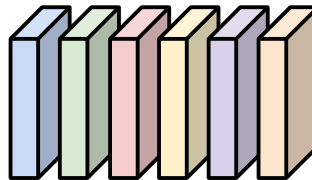


Also 6-dim bias vector:

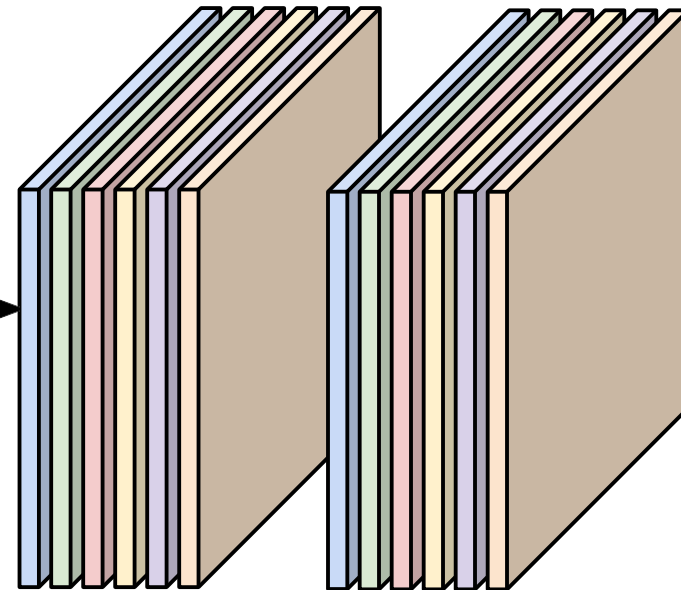


Convolution
Layer

6x3x5x5
filters

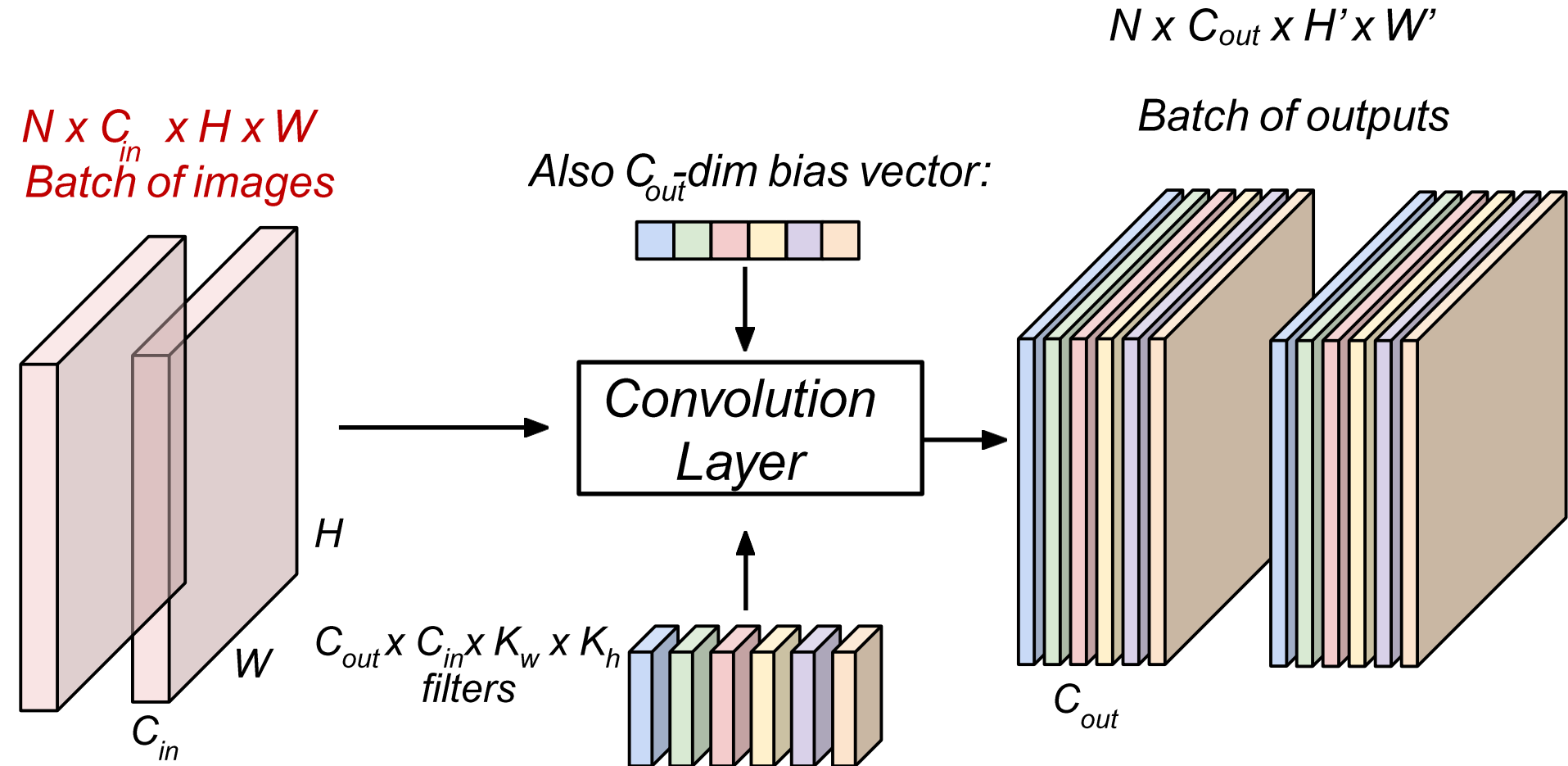


2x6x28x28 Batch
of outputs



Slide inspiration: Justin Johnson

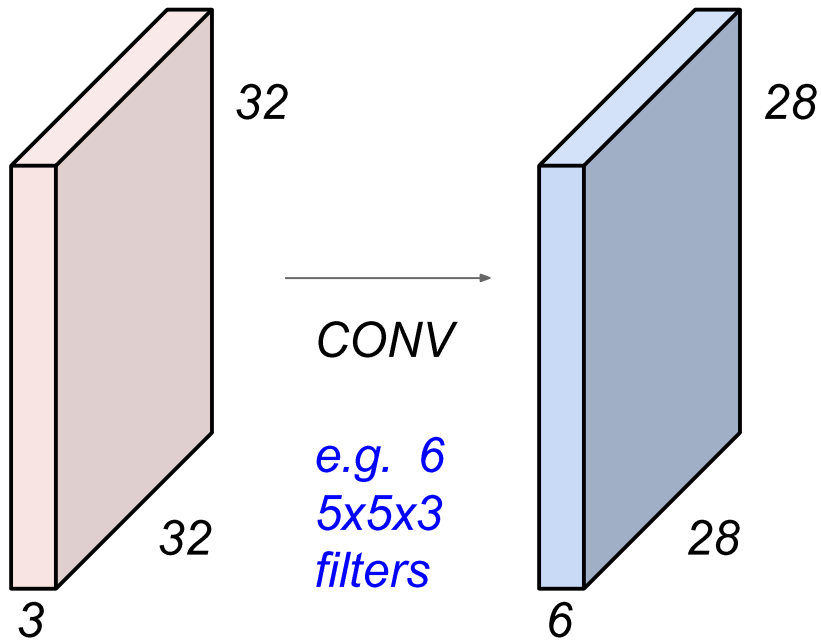
Convolution Layer



Slide inspiration: Justin Johnson

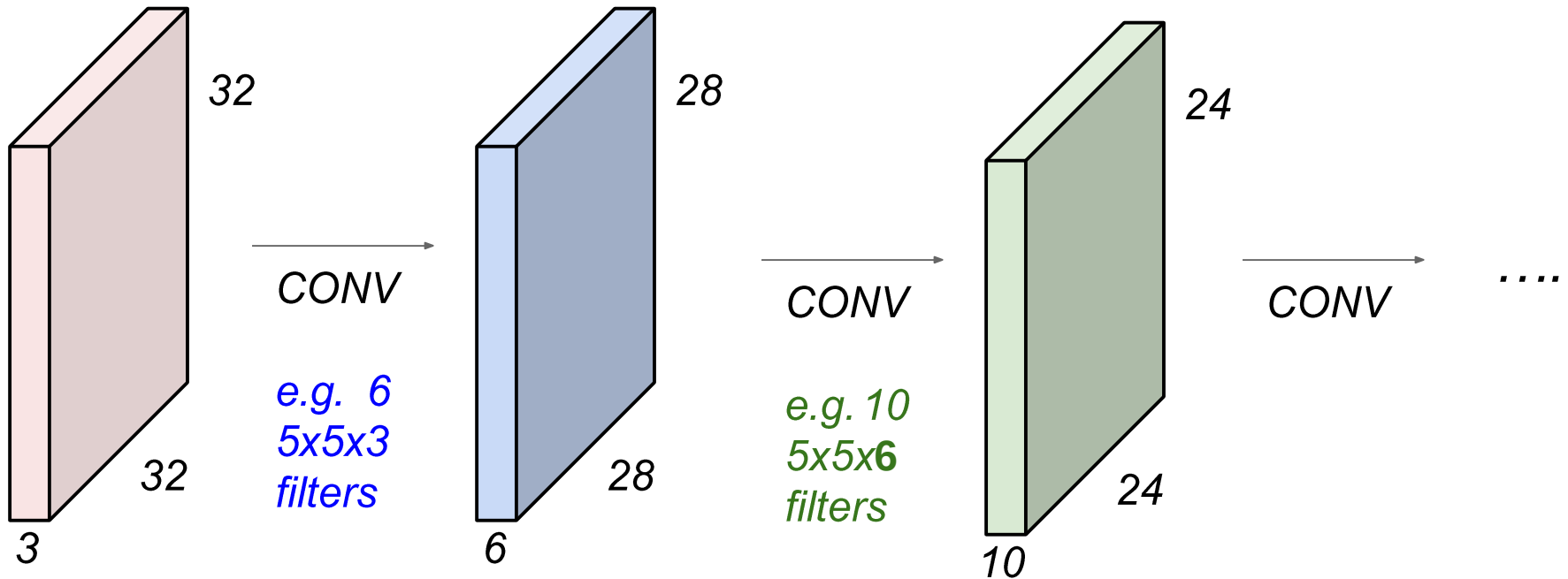


Preview: ConvNet is a sequence of Convolution Layers





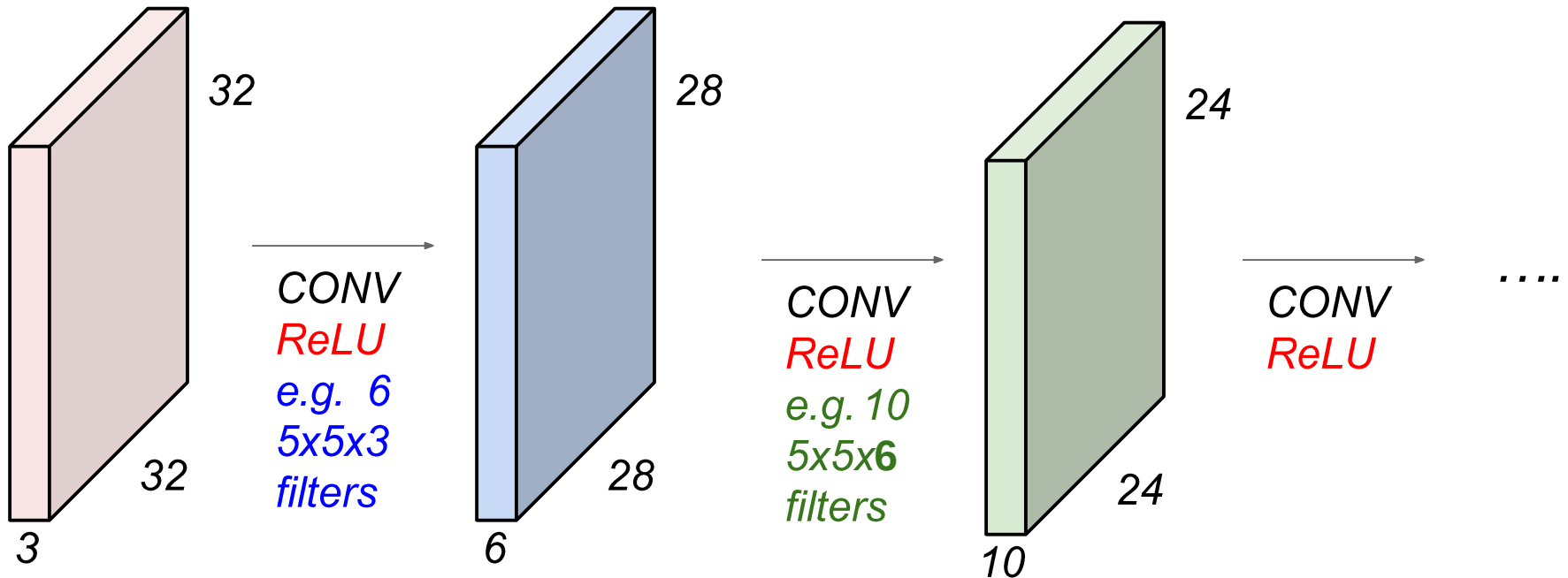
Preview: ConvNet is a sequence of Convolution Layers



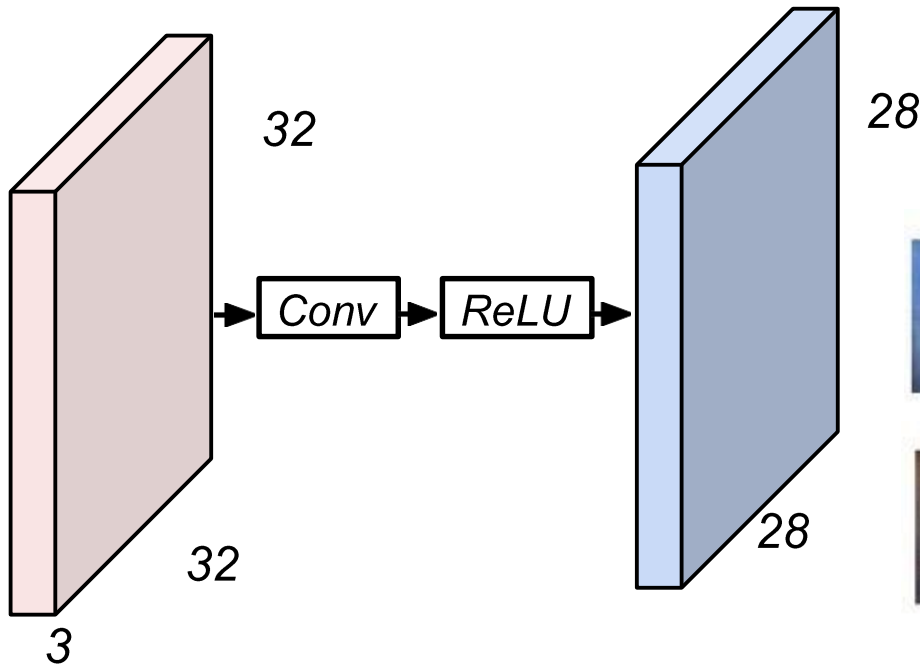


Preview: ConvNet is a sequence of Convolution Layers

ConvNet is a sequence of Convolution Layers,
interspersed with activation functions



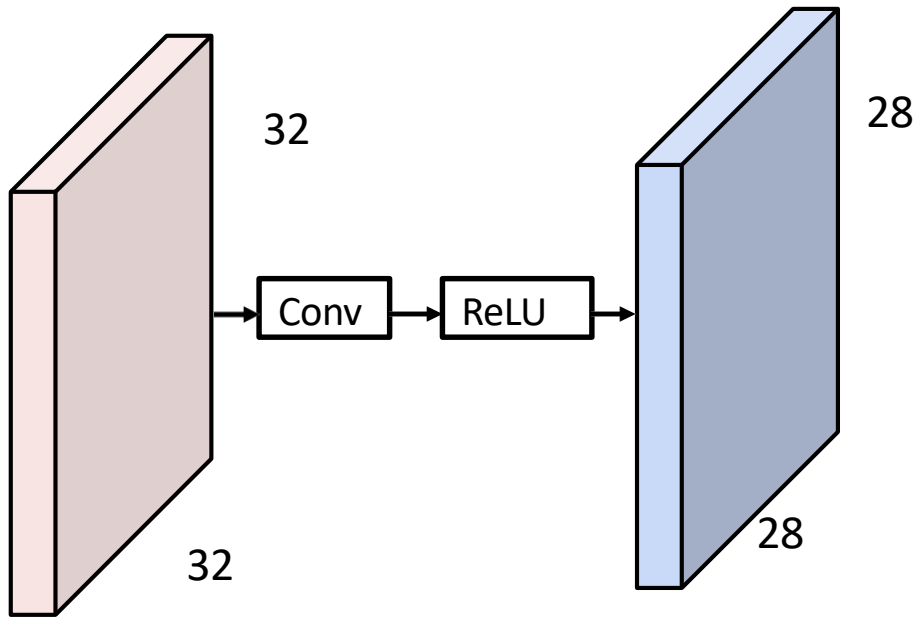
Preview: What do convolutional filters learn?



Linear classifier: One template per class



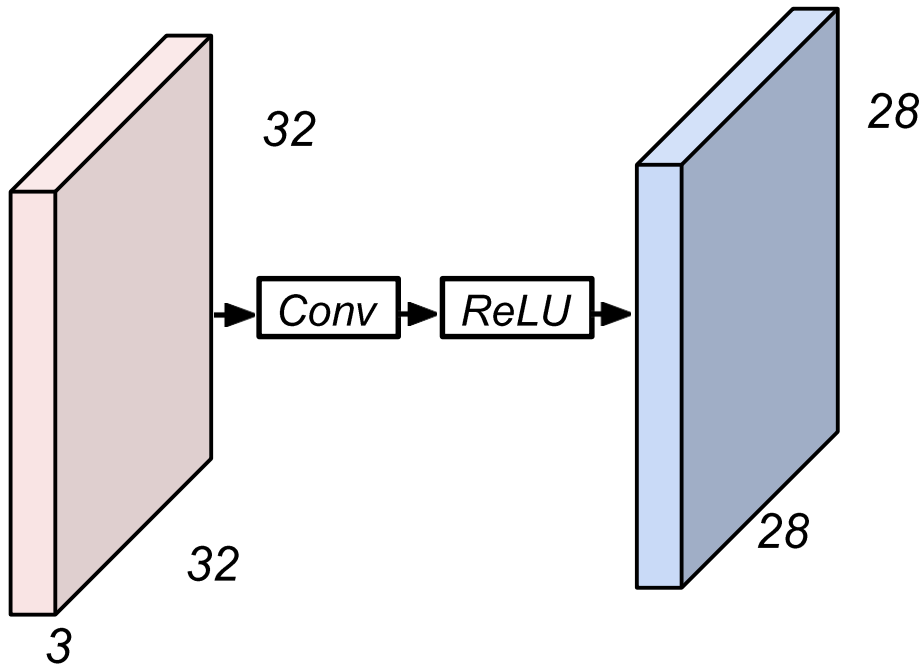
Preview: What do convolutional filters learn?



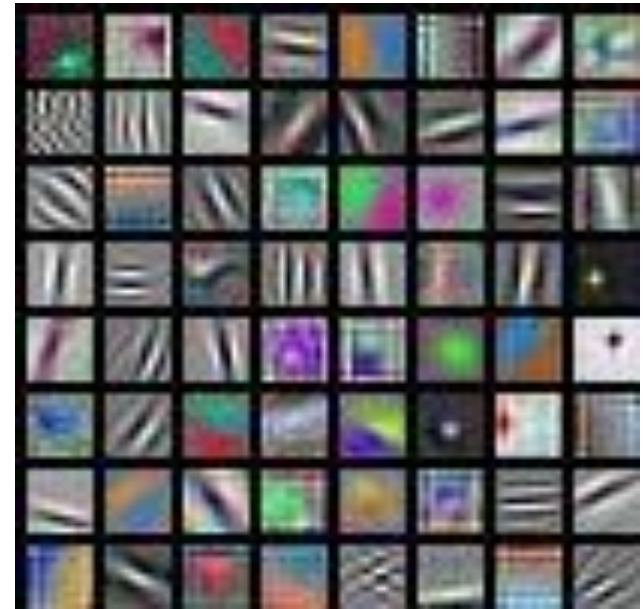
MLP: Bank of whole-image templates



Preview: What do convolutional filters learn?

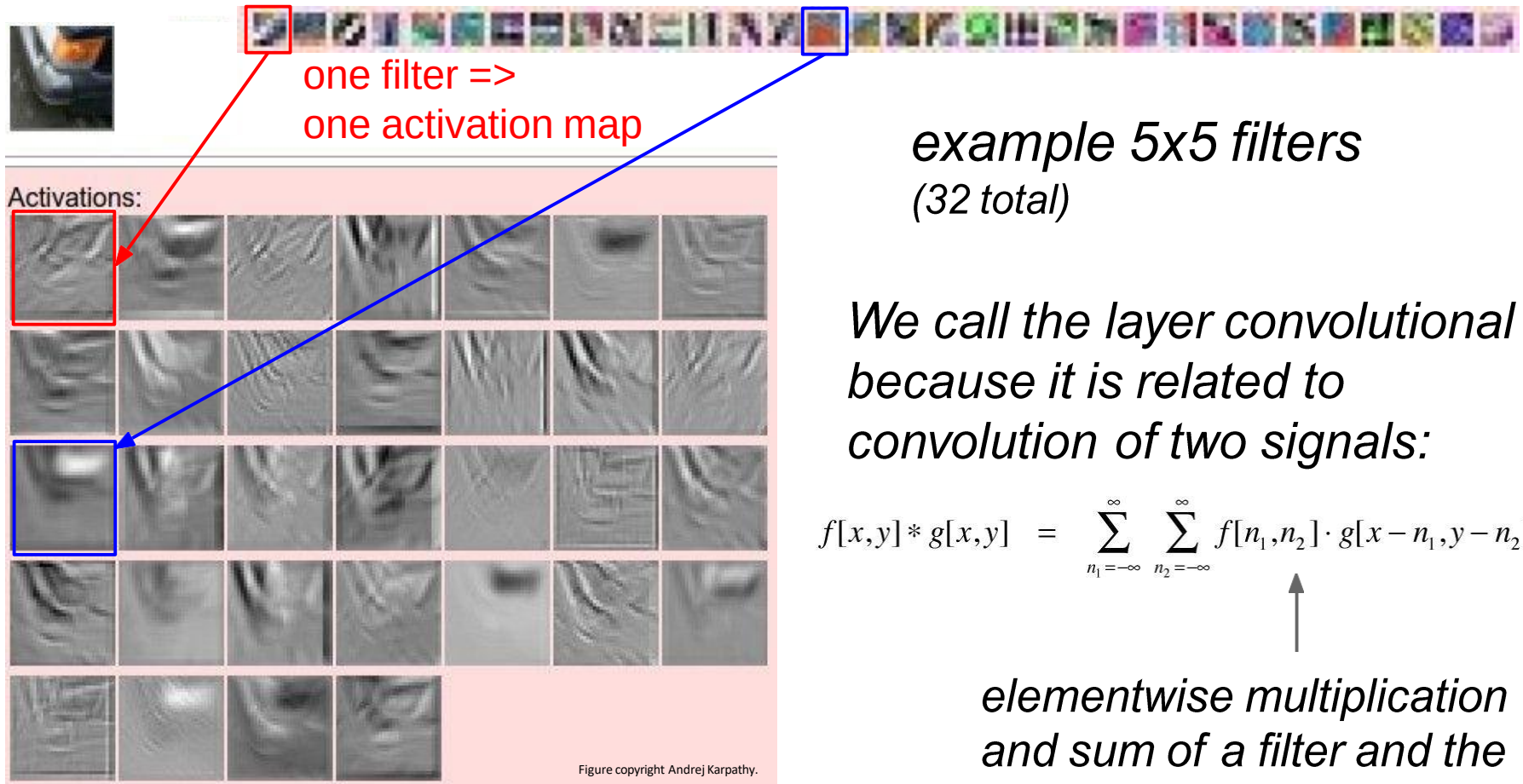


*First-layer conv filters: local image templates
(Often learns oriented edges, opposing colors)*

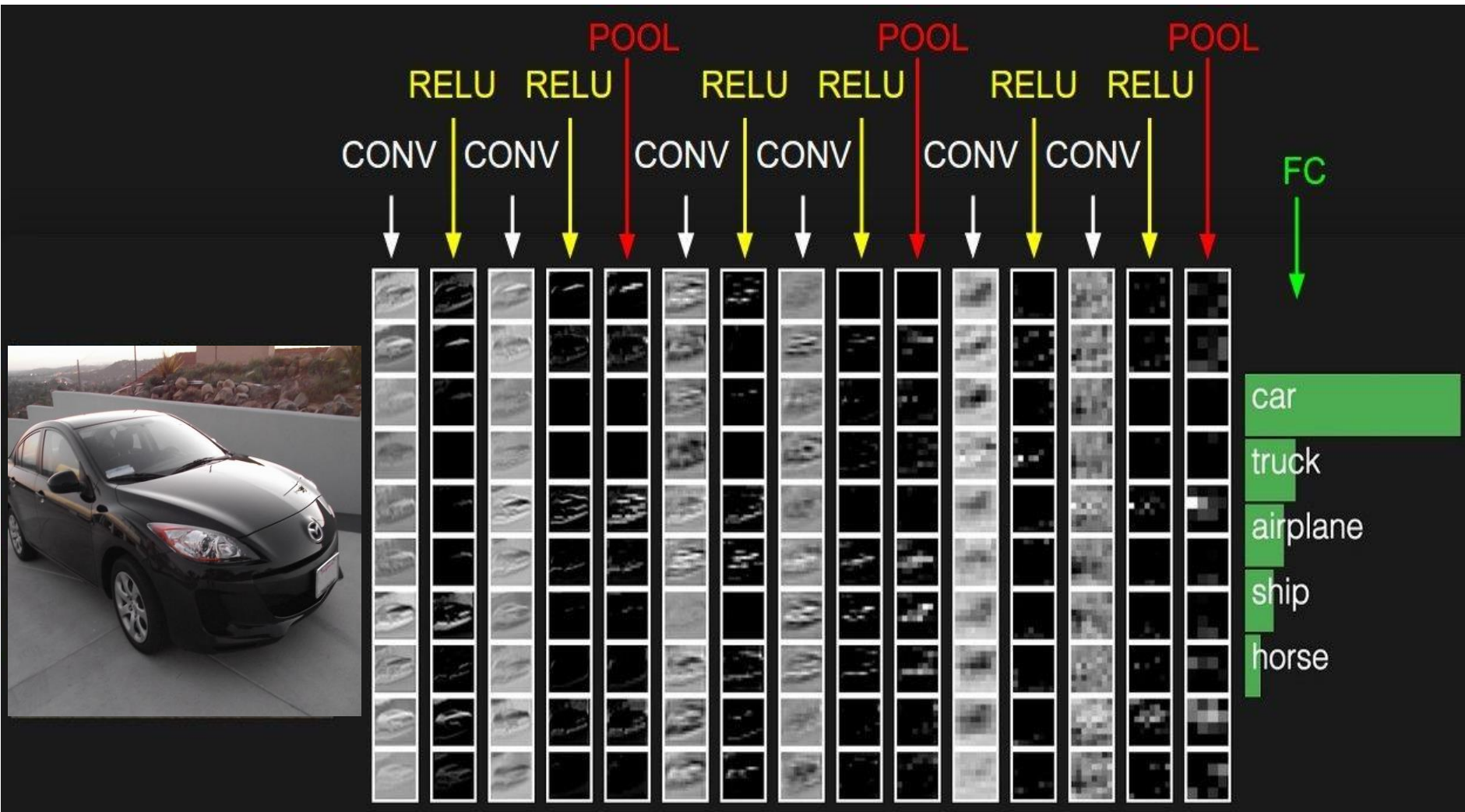


AlexNet: 64 filters, each 3x11x11

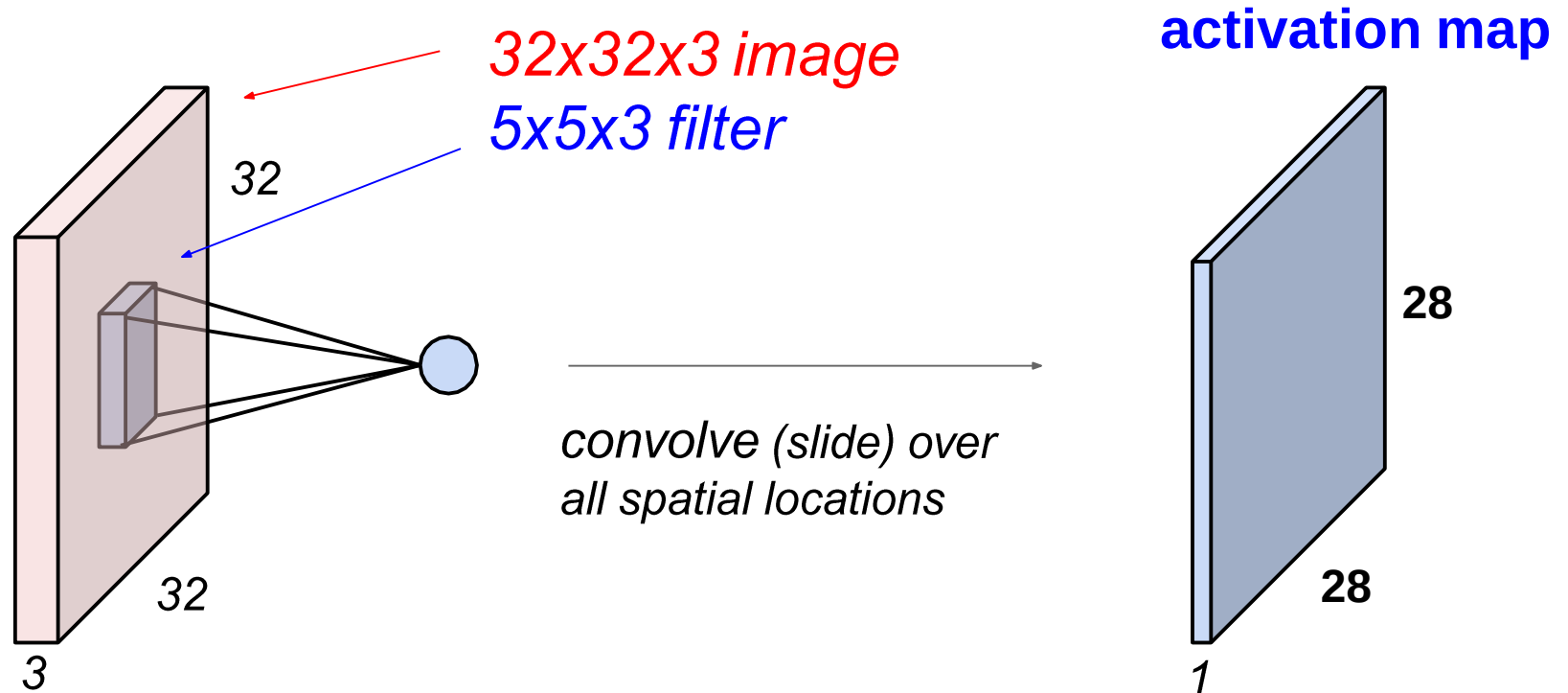
Preview



Preview

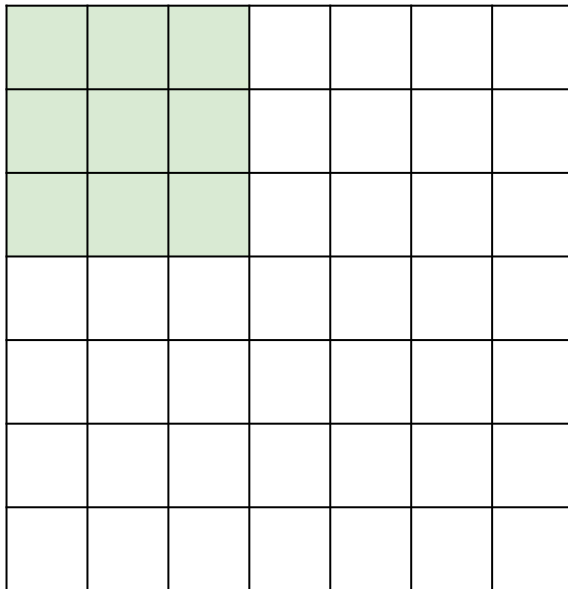


A closer look at spatial dimensions:



A closer look at spatial dimensions:

7

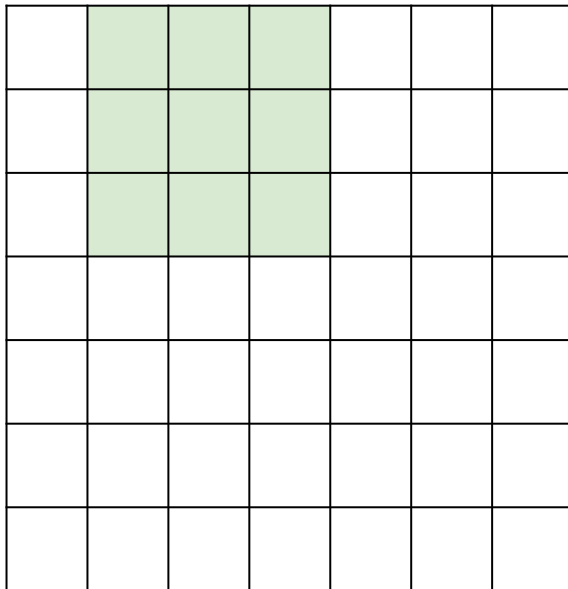


7

*7x7 input (spatially)
assume 3x3 filter*

A closer look at spatial dimensions:

7

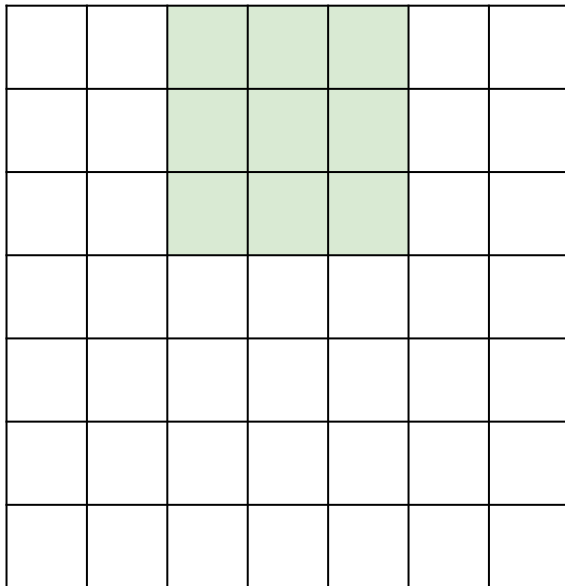


7

*7x7 input (spatially)
assume 3x3 filter*

A closer look at spatial dimensions:

7



7

*7x7 input (spatially)
assume 3x3 filter*

A closer look at spatial dimensions:

7

7

*7x7 input (spatially)
assume 3x3 filter*

A closer look at spatial dimensions:

7

7

*7x7 input (spatially)
assume 3x3 filter*

=> 5x5 output

A closer look at spatial dimensions:

7

7

7x7 input (spatially)
assume 3x3 filter
applied with stride 2

A closer look at spatial dimensions:

7

7

7x7 input (spatially)
assume 3x3 filter
applied with stride 2

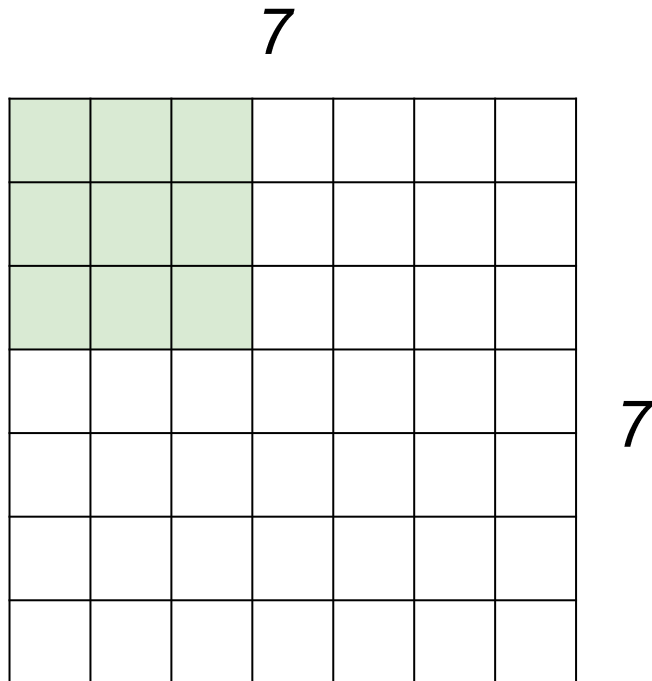
A closer look at spatial dimensions:

7

7

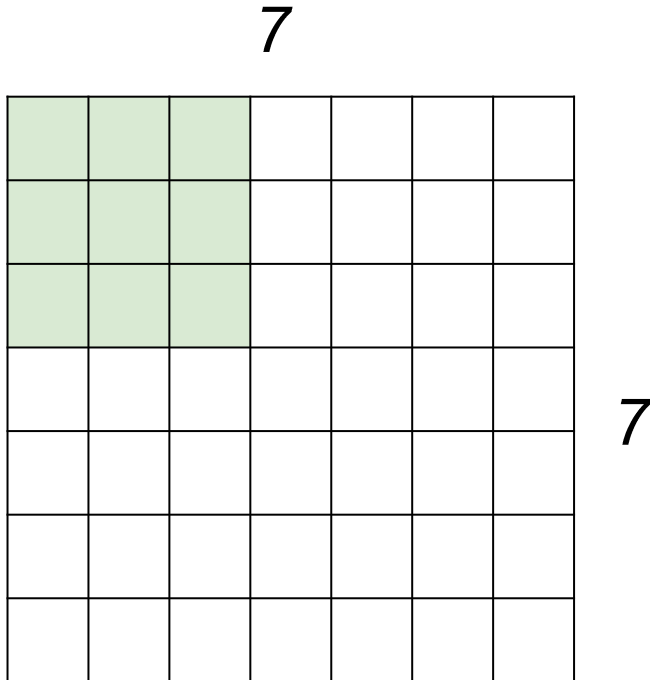
7x7 input (spatially)
assume 3x3 filter
applied with stride 2
=> 3x3 output!

A closer look at spatial dimensions:



7x7 input (spatially)
assume 3x3 filter
applied with stride 3?

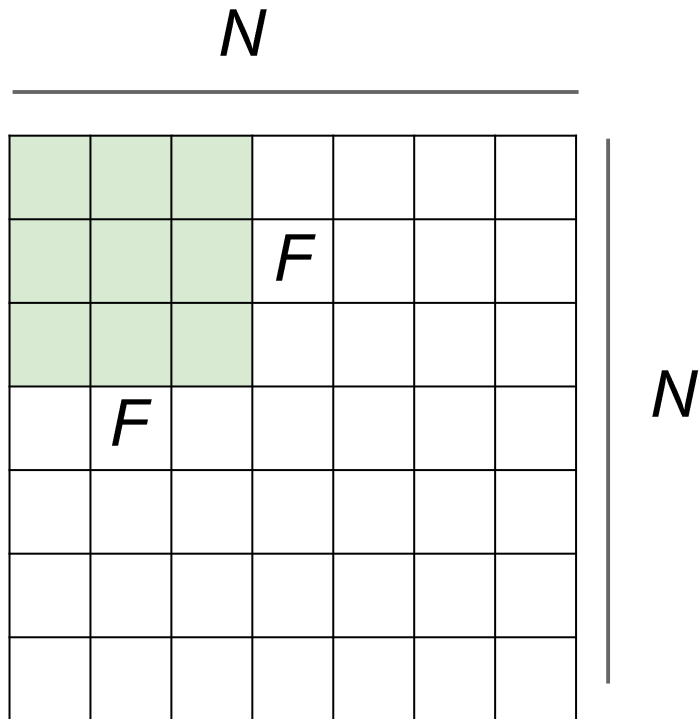
A closer look at spatial dimensions:



7x7 input (spatially)
assume 3x3 filter
applied with stride 3?

doesn't fit!
cannot apply 3x3 filter on
7x7 input with stride 3.

A closer look at spatial dimensions:



Output size:
 $(N - F) / \text{stride} + 1$

e.g. $N = 7, F = 3$:

stride 1 $\Rightarrow (7 - 3) / 1 + 1 = 5$

stride 2 $\Rightarrow (7 - 3) / 2 + 1 = 3$

stride 3 $\Rightarrow (7 - 3) / 3 + 1 = 2.33 \therefore$

In practice: Common to zero pad the border

0	0	0	0	0	0			
0								
0								
0								
0								

e.g. input 7x7

3x3 filter, applied with **stride 1**

pad with 1 pixel border

=> what is the output?

recall : $(N - F) / \text{stride} + 1$

In practice: Common to zero pad the border

0	0	0	0	0	0			
0								
0								
0								
0								

e.g. input 7x7

3x3 filter, applied with **stride 1**
pad with 1 pixel border =>
what is the output?

7x7 output!

recall : $(N + 2P - F) / \text{stride} + 1$

In practice: Common to zero pad the border

0	0	0	0	0	0			
0								
0								
0								
0								

e.g. input 7x7

3x3 filter, applied with **stride 1**

pad with 1 pixel border => what is the output?

7x7 output!

in general, common to see CONV layers with stride 1, filters of size $F \times F$, and zero-padding with $(F-1)/2$. (will preserve size spatially)

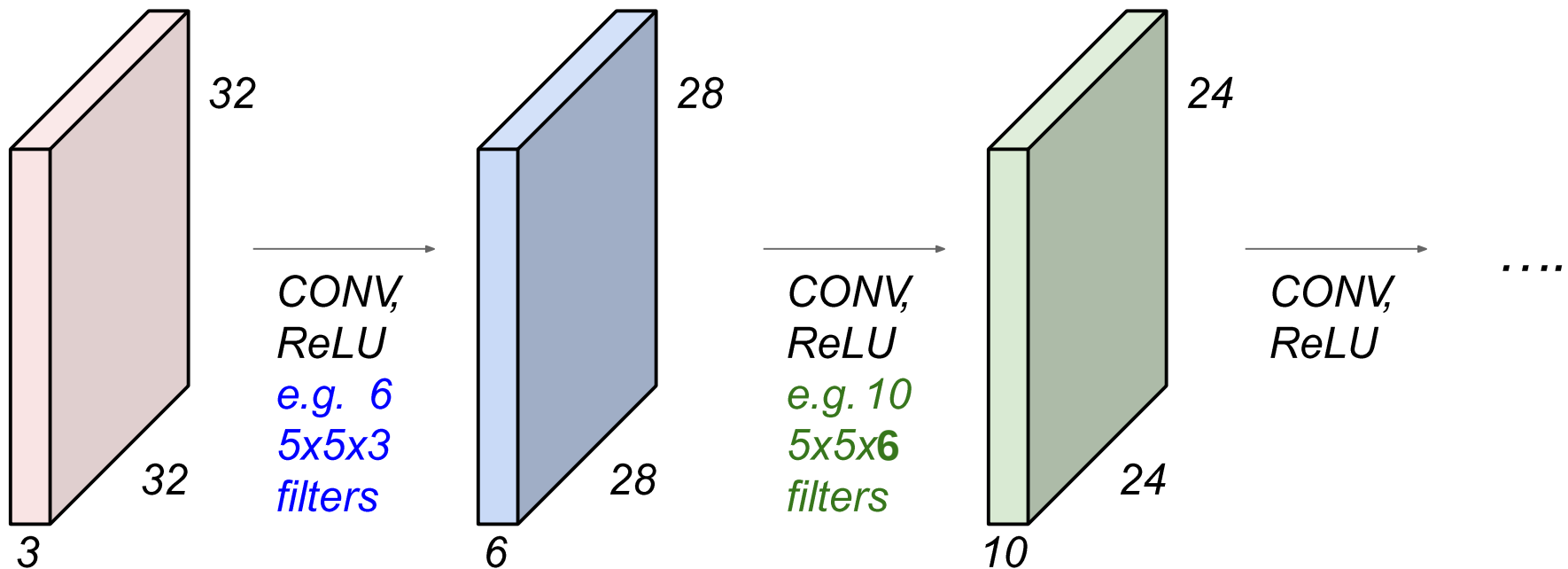
e.g. $F = 3$ => zero pad with 1

$F = 5$ => zero pad with 2

$F = 7$ => zero pad with 3

Remember back to...

E.g. 32x32 input convolved repeatedly with 5x5 filters shrinks volumes spatially! (32 -> 28 -> 24 ...). Shrinking too fast is not good, doesn't work well.

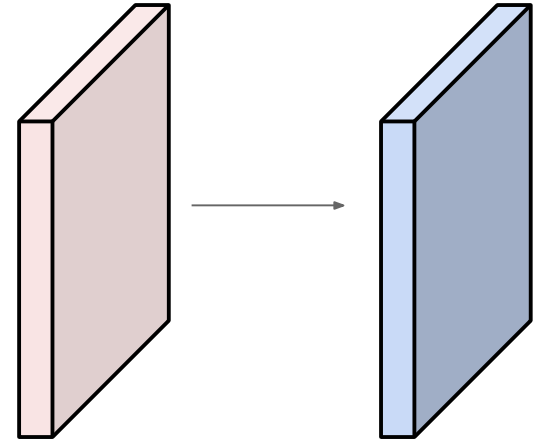


Examples time:

*Input volume: **32x32x3***

10 5x5 filters with stride 1, pad 2

Output volume size: ?



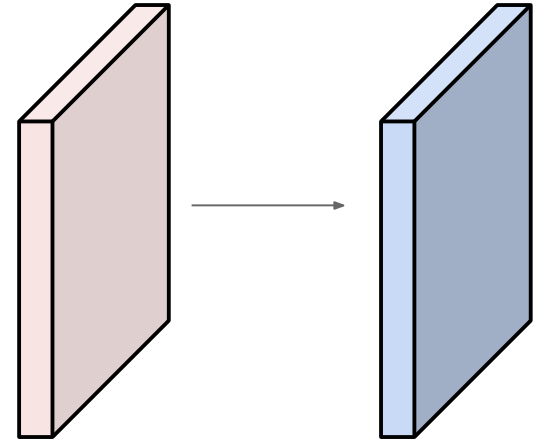
Examples time:

*Input volume: **32x32x3***

10** **5x5** filters with stride **1**, pad **2

Output volume size:

*(**32**+2***2**-**5**)/**1**+1 = 32 spatially, so
32x32x10*

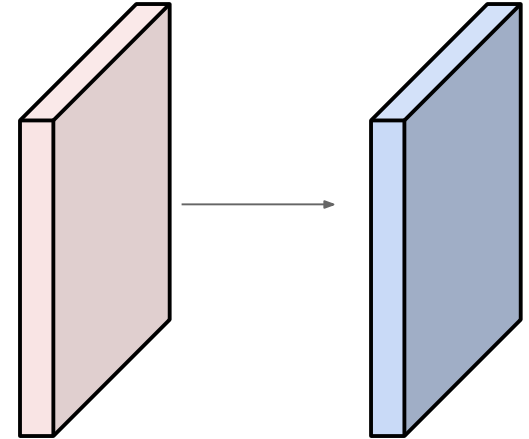


Examples time:

*Input volume: **32x32x3***

10 5x5 filters with stride 1, pad 2

Number of parameters in this layer?

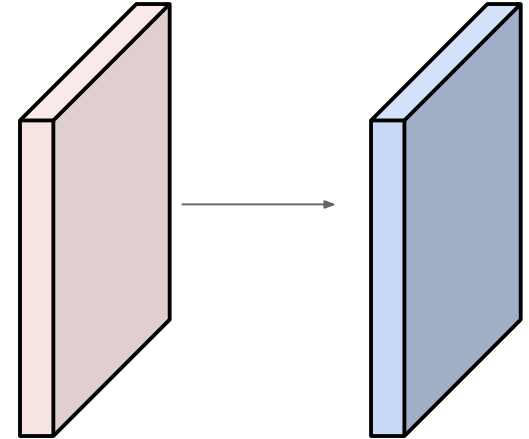


Examples time:

*Input volume: **32x32x3***

***10** **5x5** filters with stride 1, pad 2*

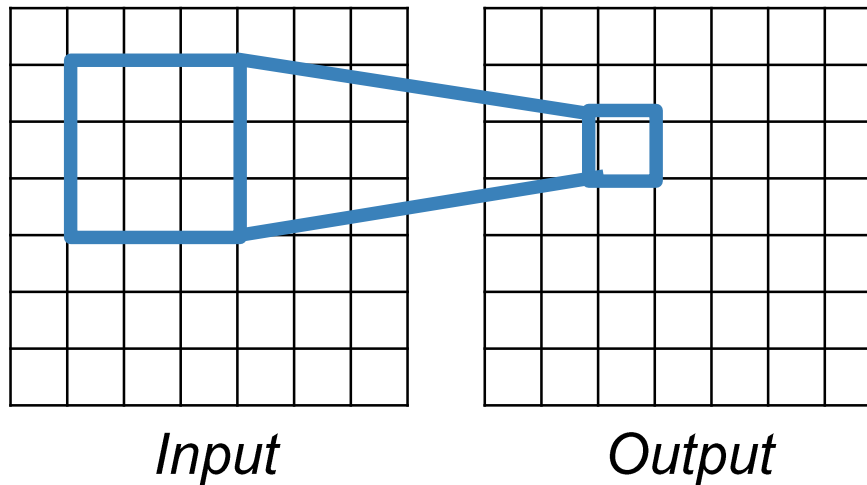
Number of parameters in this layer?



*each filter has $5*5*3 + 1 = 76$ params
 $\Rightarrow 76*10 = 760$*

Receptive Fields

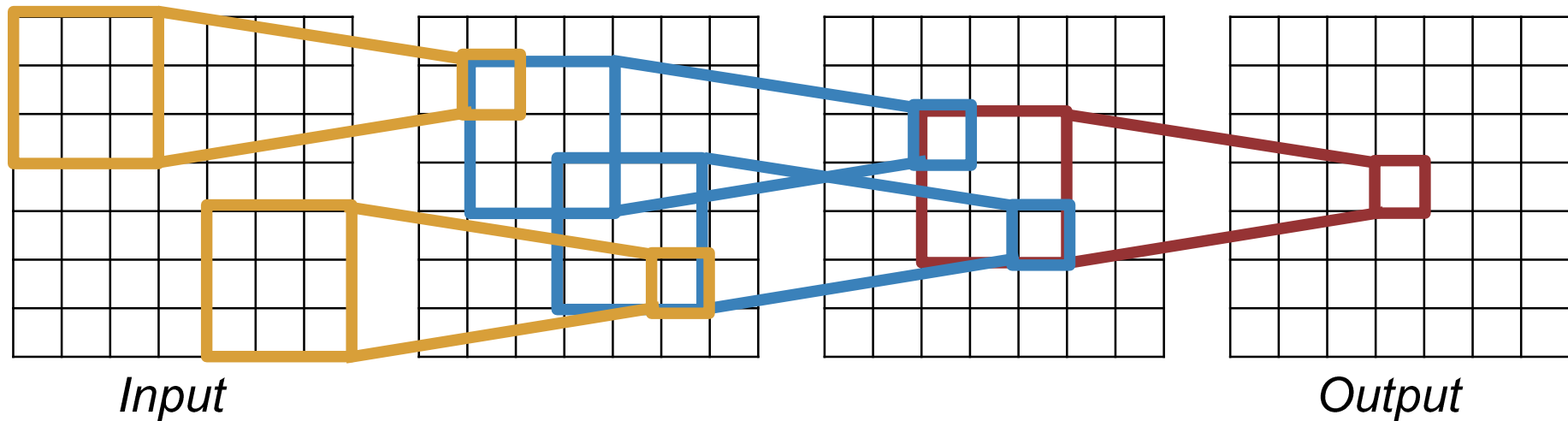
*For convolution with kernel size K , each element in the output depends on a $K \times K$ **receptive field** in the input*



Slide inspiration: Justin Johnson

Receptive Fields

*Each successive convolution adds $K - 1$ to the receptive field size
With L layers the receptive field size is $1 + L * (K - 1)$*

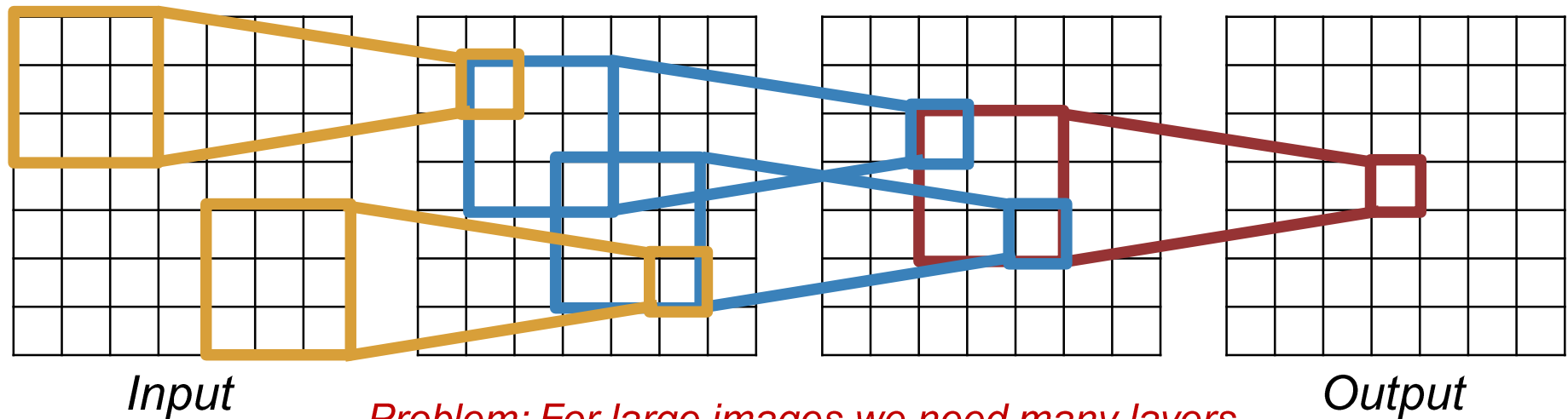


Be careful – “receptive field in the input” vs. “receptive field in the previous layer”

Slide inspiration: Justin Johnson

Receptive Fields

*Each successive convolution adds $K - 1$ to the receptive field size
With L layers the receptive field size is $1 + L * (K - 1)$*

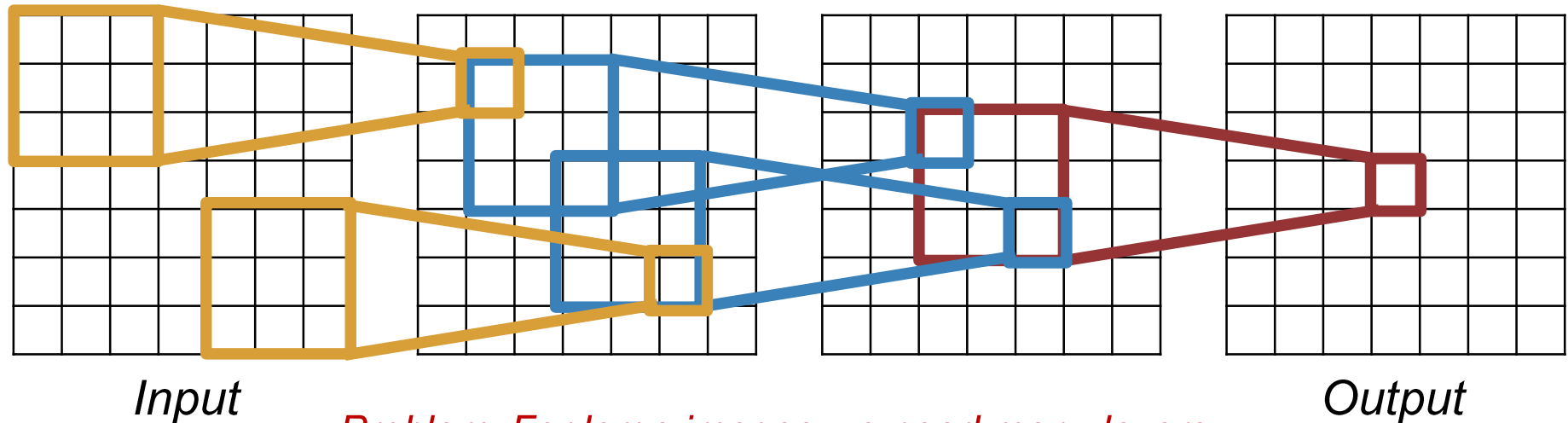


*Problem: For large images we need many layers
for each output to “see” the whole image*

Slide inspiration: Justin Johnson

Receptive Fields

*Each successive convolution adds $K - 1$ to the receptive field size With L layers the receptive field size is $1 + L * (K - 1)$*



Problem: For large images we need many layers for each output to “see” the whole image

Solution: Downsample inside the network

Slide inspiration: Justin Johnson

Solution: Strided Convolution

7

7

7x7 input (spatially)
assume 3x3 filter
applied with stride 2

Solution: Strided Convolution

7

7

7x7 input (spatially)
assume 3x3 filter
applied with stride 2

=> 3x3 output!



Convolution layer: summary

Let's assume input is $W_1 \times H_1 \times C$ Conv layer needs 4 hyperparameters:

- *Number of filters **K***
- *The filter size **F***
- *The stride **S***
- *The zero padding **P***

This will produce an output of $W_2 \times H_2 \times K$ where:

- $W_2 = (W_1 - F + 2P) / S + 1$
- $H_2 = (H_1 - F + 2P) / S + 1$

Number of parameters: F^2CK and K biases



Convolution layer: summary

Let's assume input is $W_1 \times H_1 \times C$ Conv layer needs 4 hyperparameters:

- Number of filters **K**
- The filter size **F**
- The stride **S**
- The zero padding **P**

Common settings:

K = (powers of 2, e.g. 32, 64, 128, 512)

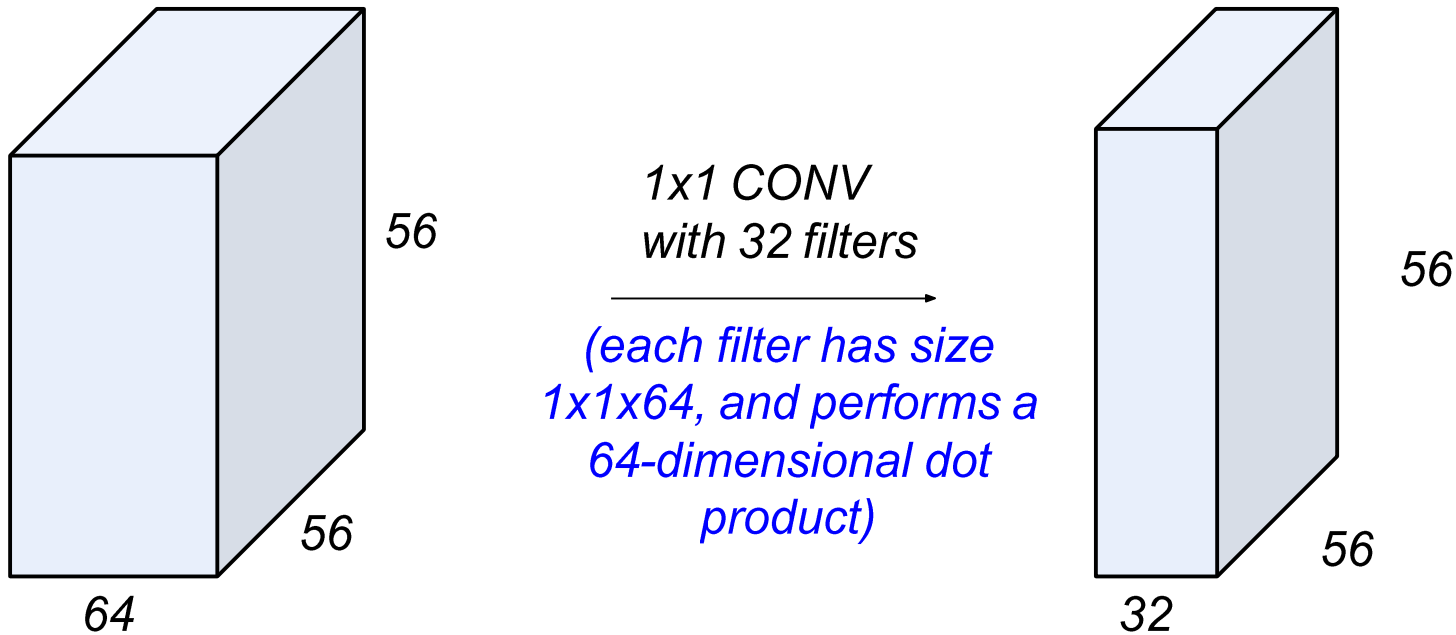
- *F = 3, S = 1, P = 1*
- *F = 5, S = 1, P = 2*
- *F = 5, S = 2, P = ? (whatever fits)*
- *F = 1, S = 1, P = 0*

This will produce an output of $W_2 \times H_2 \times K$ where:

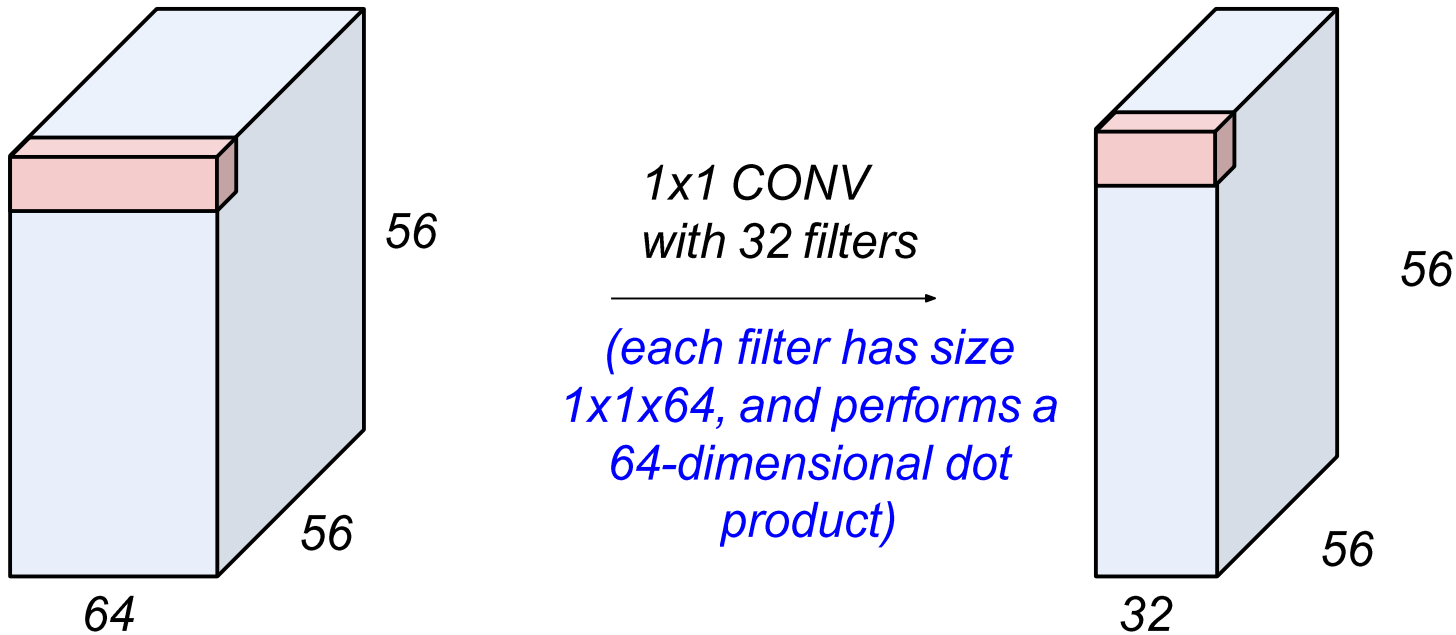
- $W_2 = (W_1 - F + 2P) / S + 1$
- $H_2 = (H_1 - F + 2P) / S + 1$

Number of parameters: F^2CK and K biases

1x1 convolution layers make perfect sense



1x1 convolution layers make perfect sense



Example: CONV layer in PyTorch

Conv2d

```
CLASS torch.nn.Conv2d(in_channels, out_channels, kernel_size, stride=1, padding=0, dilation=1, groups=1, bias=True)
```

[\[SOURCE\]](#)

Applies a 2D convolution over an input signal composed of several input planes.

In the simplest case, the output value of the layer with input size (N, C_{in}, H, W) and output $(N, C_{out}, H_{out}, W_{out})$ can be precisely described as:

$$\text{out}(N_i, C_{out_j}) = \text{bias}(C_{out_j}) + \sum_{k=0}^{C_{in}-1} \text{weight}(C_{out_j}, k) \star \text{input}(N_i, k)$$

where $\star$ is the valid 2D **cross-correlation** operator, N is a batch size, C denotes a number of channels, H is a height of input planes in pixels, and W is width in pixels.

- `stride` controls the stride for the cross-correlation, a single number or a tuple.
- `padding` controls the amount of implicit zero-paddings on both sides for `padding` number of points for each dimension.
- `dilation` controls the spacing between the kernel points; also known as the à trous algorithm. It is harder to describe, but this [link](#) has a nice visualization of what `dilation` does.
- `groups` controls the connections between inputs and outputs. `in_channels` and `out_channels` must both be divisible by `groups`. For example,
 - At `groups=1`, all inputs are convolved to all outputs.
 - At `groups=2`, the operation becomes equivalent to having two conv layers side by side, each seeing half the input channels, and producing half the output channels, and both subsequently concatenated.
 - At `groups= in_channels`, each input channel is convolved with its own set of filters, of size: $\left\lfloor \frac{C_{out}}{C_{in}} \right\rfloor$.

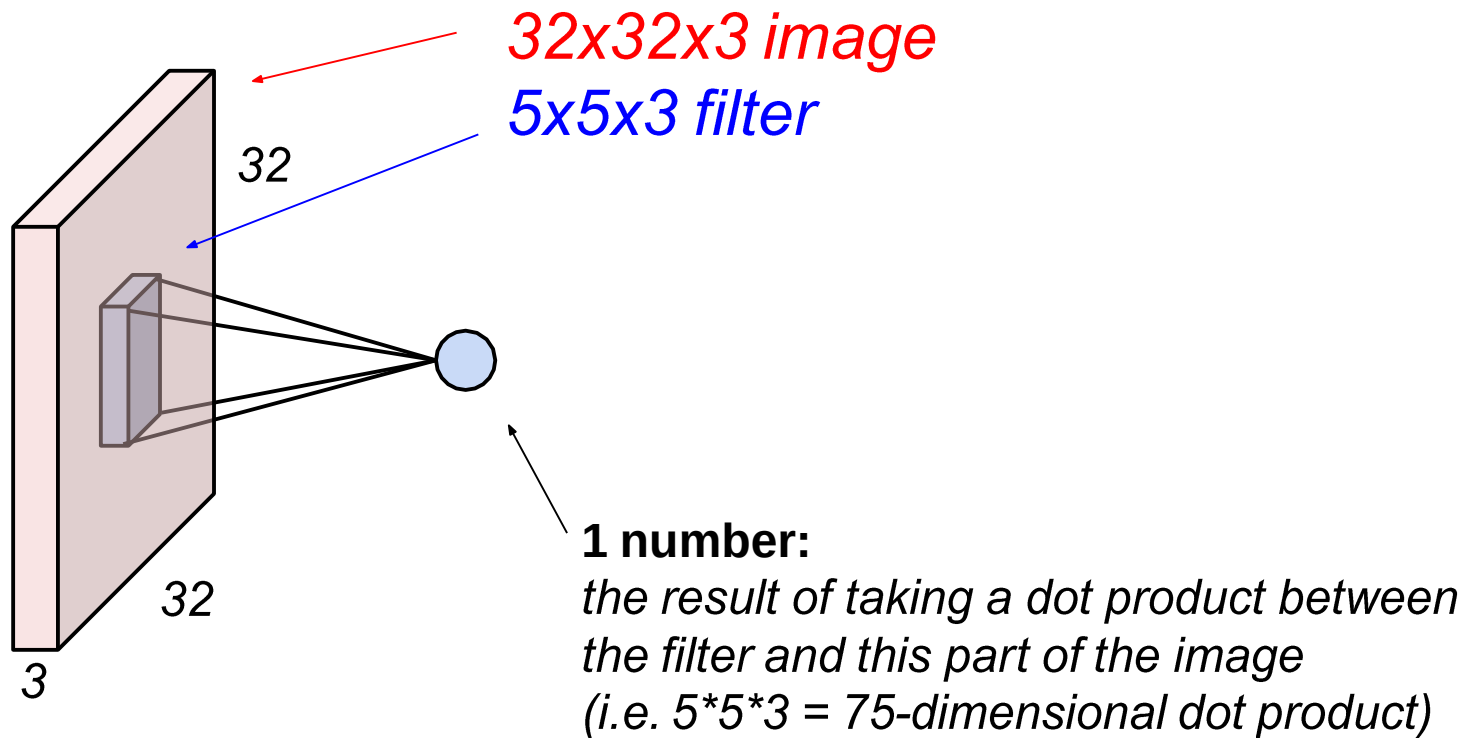
The parameters `kernel_size`, `stride`, `padding`, `dilation` can either be:

- a single `int` – in which case the same value is used for the height and width dimension
- a `tuple` of two ints – in which case, the first `int` is used for the height dimension, and the second `int` for the width dimension

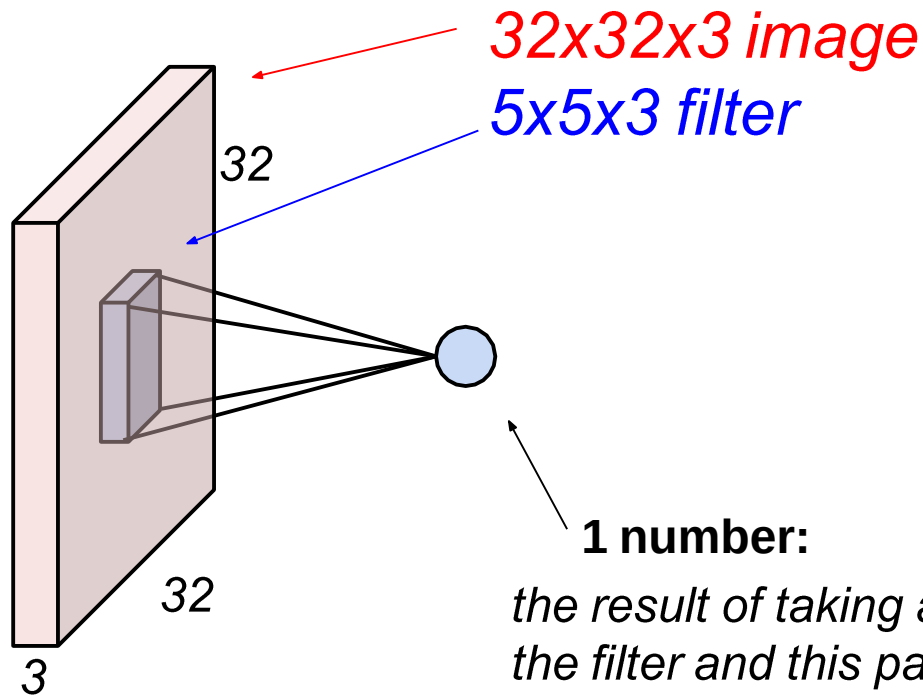
Conv layer needs 4 hyperparameters:

- *Number of filters **K***
- *The filter size **F***
- *The stride **S***
- *The zero padding **P***

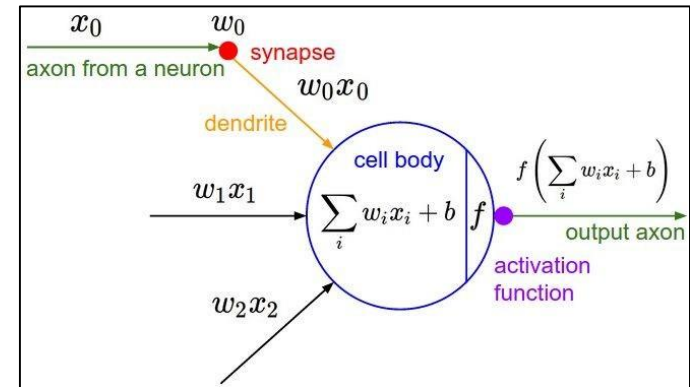
The brain/neuron view of CONV Layer



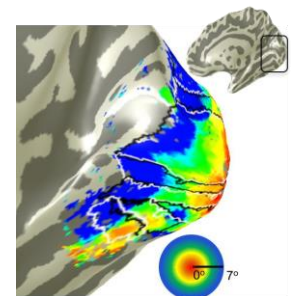
The brain/neuron view of CONV Layer



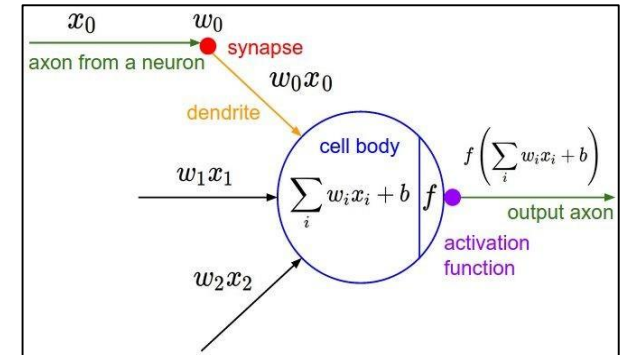
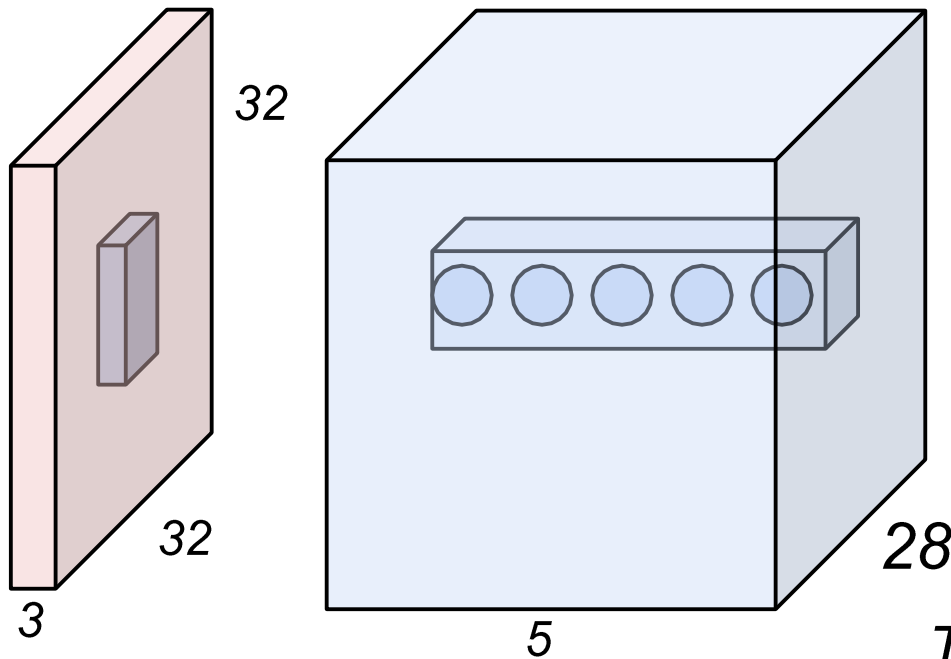
the result of taking a dot product between the filter and this part of the image (i.e. $5 \times 5 \times 3 = 75$ -dimensional dot product)



It's just a neuron with local connectivity...



The brain/neuron view of CONV Layer

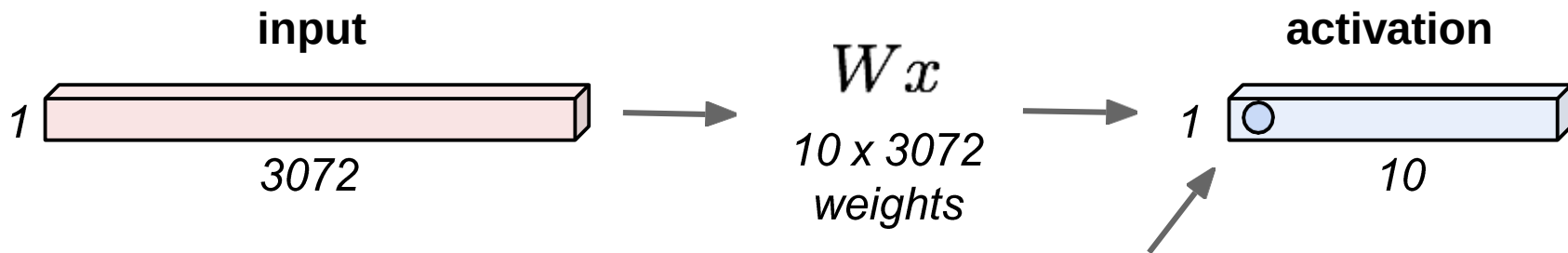


*E.g. with 5 filters,
CONV layer consists of
neurons arranged in a 3D
grid (28x28x5)*

*There will be 5 different neurons
all looking at the same region in
the input volume*

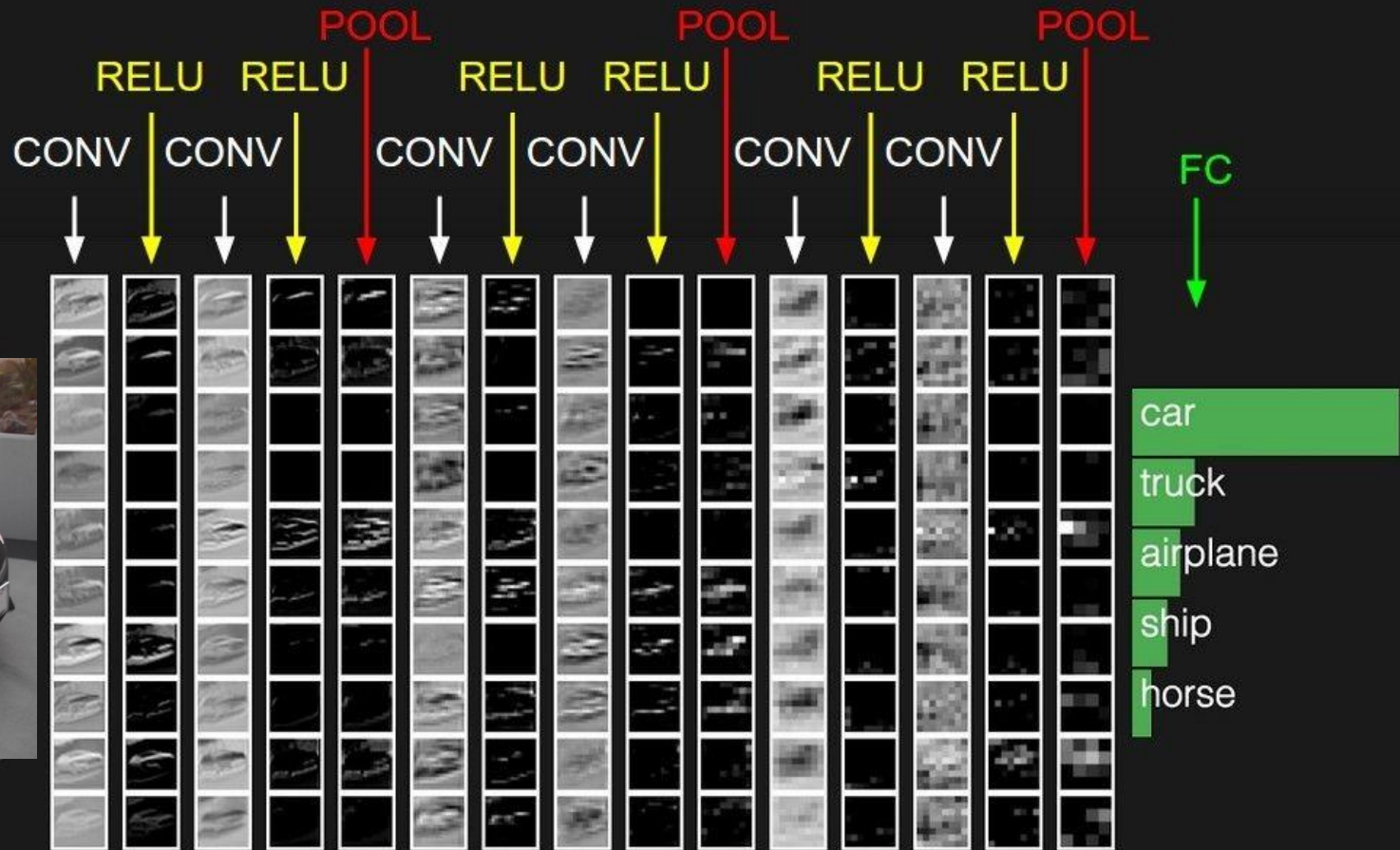
Reminder: Fully Connected Layer

32x32x3 image -> stretch to 3072 x 1



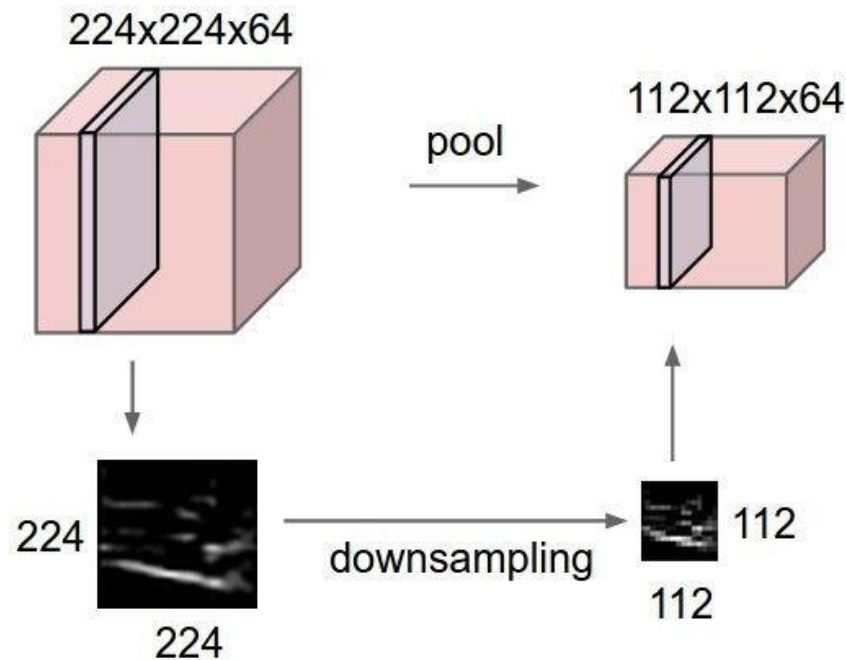
Each neuron looks at the full input volume

1 number:
the result of taking a dot product between a row of W and the input (a 3072-dimensional dot product)

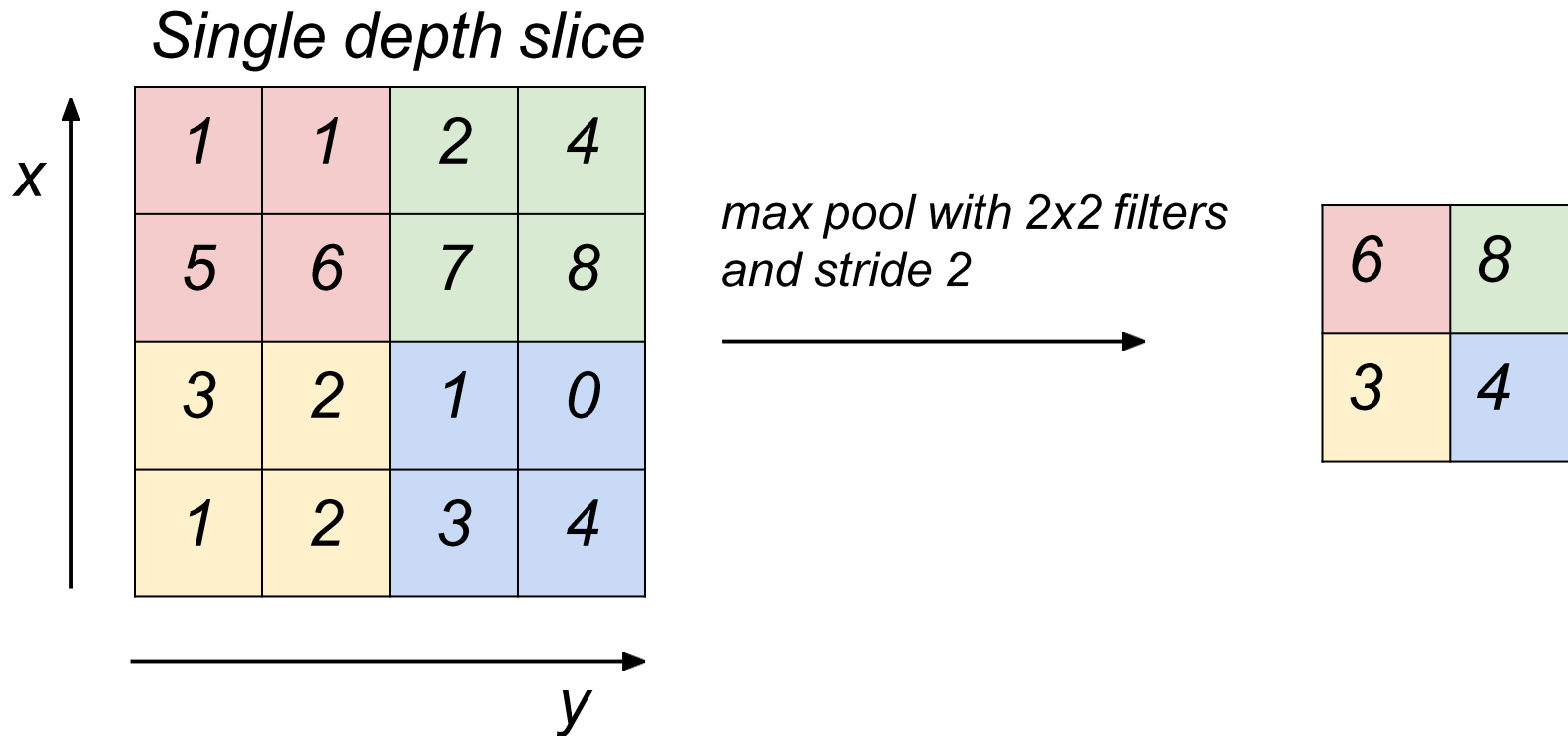


Pooling layer

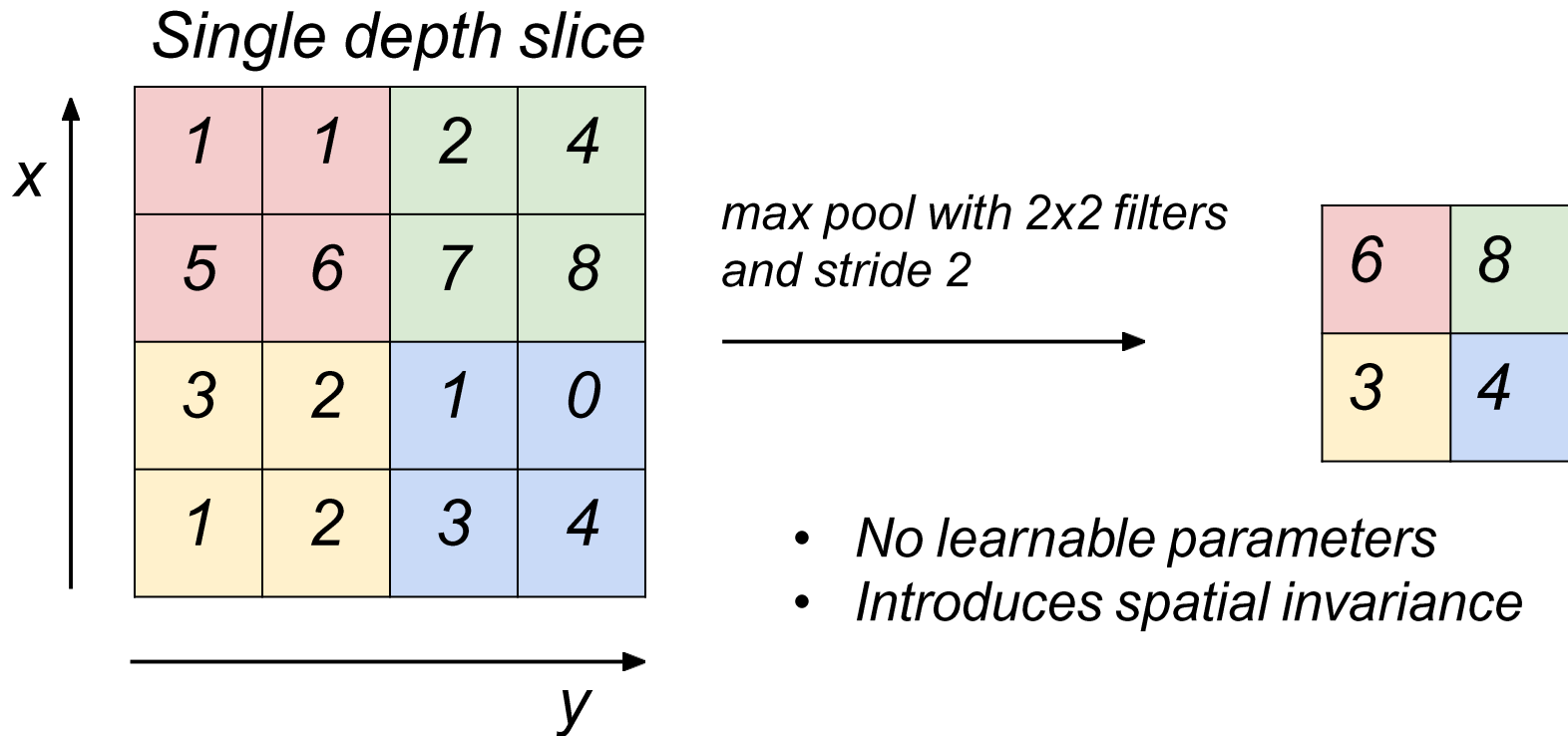
- *makes the representations smaller and more manageable*
- *operates over each activation map independently*



MAX POOLING



MAX POOLING



Pooling layer: summary

Let's assume input is $W_1 \times H_1 \times C$ Conv layer needs 2 hyperparameters:

- *The spatial extent **F***
- *The stride **S***

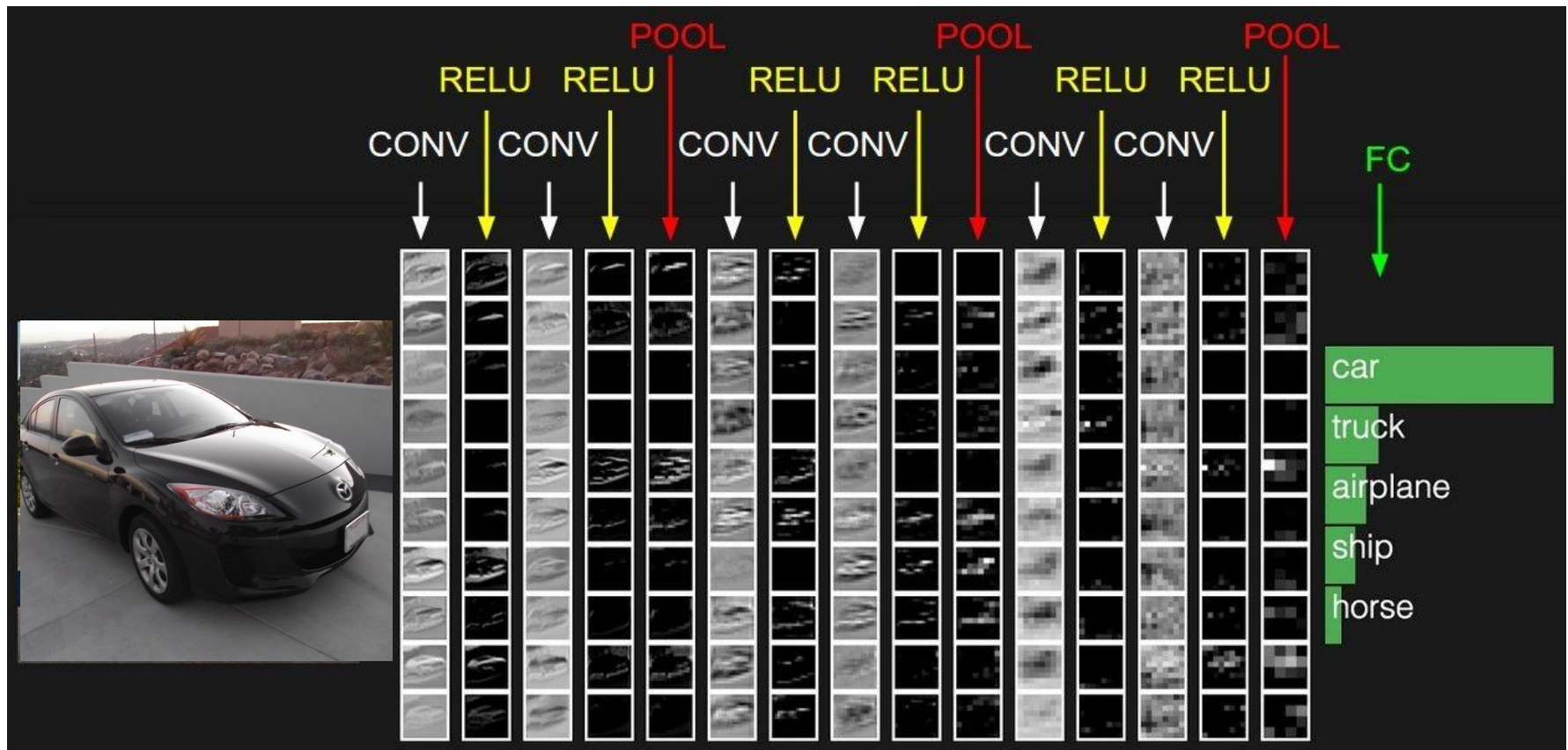
This will produce an output of $W_2 \times H_2 \times C$ where:

- $W_2 = (W_1 - F) / S + 1$
- $H_2 = (H_1 - F) / S + 1$

Number of parameters: 0

Fully Connected Layer (FC layer)

Contains neurons that connect to the entire input volume, as in ordinary Neural Networks



Summary

- *ConvNets stack CONV, POOL, FC layers*
- *Trend towards smaller filters and deeper architectures*
- *Trend towards getting rid of POOL/FC layers (just CONV)*
- *Historically architectures looked like*
[(CONV-RELU)*N-POOL?]*M-(FC-RELU)*K, SOFTMAX
*where **N** is usually up to ~5, **M** is large, $0 \leq K \leq 2$.*
- *But recent advances have challenged this paradigm*



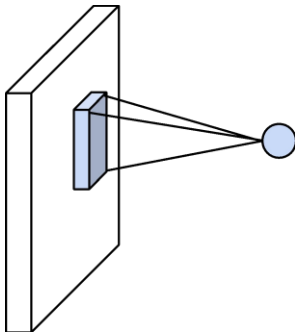
卷积神经网络

- 卷积神经网络（Convolutional Neural Networks, CNN）
 - 受生物学上感受野（Receptive Field）的机制而提出的
 - 在视觉神经系统中，一个神经元的感受野是指视网膜上的特定区域，只有这个区域内的刺激才能够激活该神经元。
- 卷积神经网络有三个结构上的特性：
 - 局部连接
 - 权重共享
 - 空间或时间上的次采样

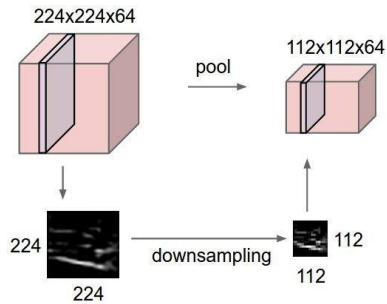


Components of CNNs

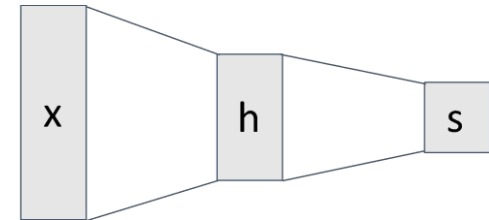
Convolution Layers



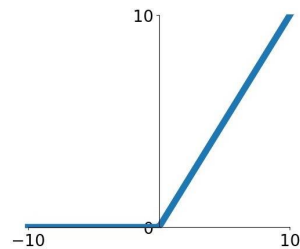
Pooling Layers



Fully-Connected Layers



Activation Function

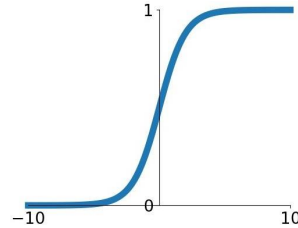


Activation Functions

Activation Functions

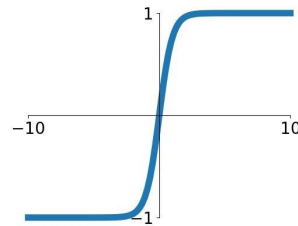
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



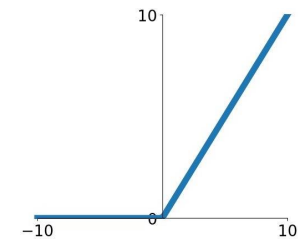
tanh

$$\tanh(x)$$



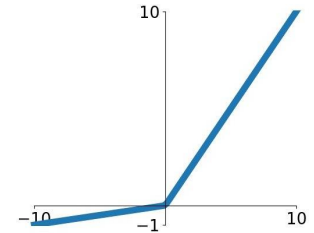
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

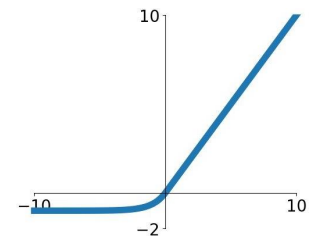


Maxout

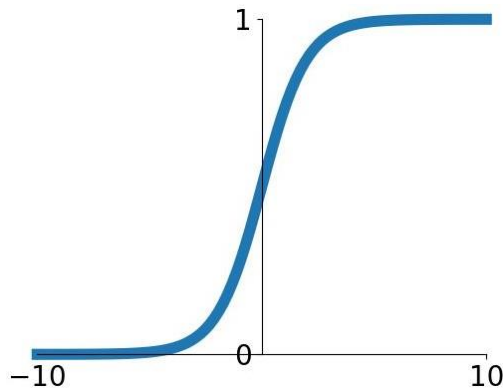
$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



Activation Functions

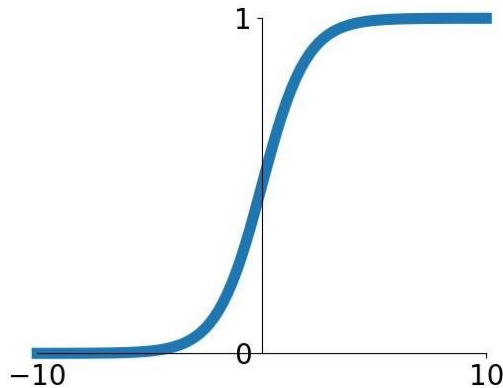


Sigmoid

$$\sigma(x) = 1 / (1 + e^{-x})$$

- Squashes numbers to range [0,1]
- Historically popular since they have nice interpretation as a saturating “firing rate” of a neuron

Activation Functions

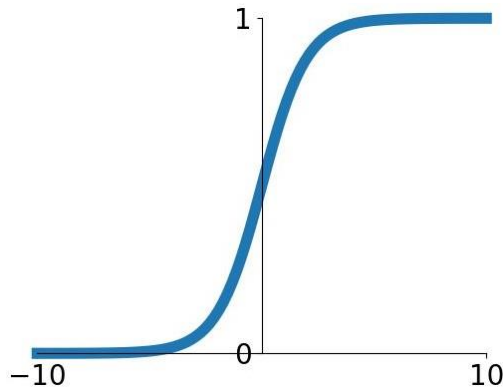


Sigmoid

$$\sigma(x) = 1/(1 + e^{-x})$$

- Squashes numbers to range $[0,1]$
- Historically popular since they have nice interpretation as a saturating “firing rate” of a neuron
- 3 *problems*:
 1. *Saturated neurons “kill” the gradients*

Activation Functions

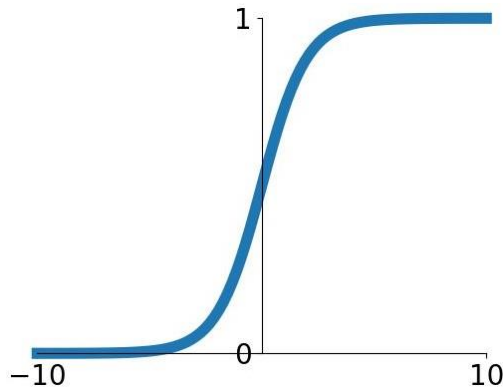


Sigmoid

$$\sigma(x) = 1/(1 + e^{-x})$$

- Squashes numbers to range $[0,1]$
- Historically popular since they have nice interpretation as a saturating “firing rate” of a neuron
- 3 *problems*:
 1. *Saturated neurons “kill” the gradients*
 2. *Sigmoid outputs are not zero-centered*

Activation Functions

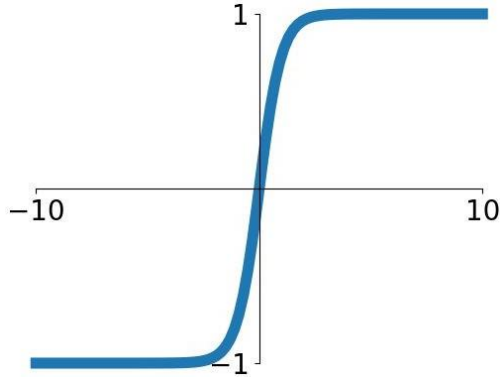


Sigmoid

$$\sigma(x) = 1/(1 + e^{-x})$$

- Squashes numbers to range $[0,1]$
- Historically popular since they have nice interpretation as a saturating “firing rate” of a neuron
- 3 problems:
 1. Saturated neurons “kill” the gradients
 2. Sigmoid outputs are not zero-centered
 3. $\exp()$ is a bit compute expensive

Activation Functions

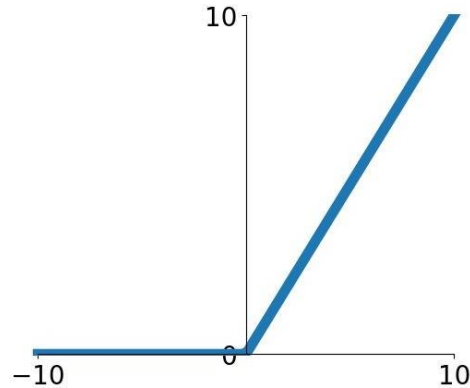


$\tanh(x)$

- Squashes numbers to range $[-1, 1]$
- zero centered (nice)
- still kills gradients when saturated :(

[LeCun et al., 1991]

Activation Functions



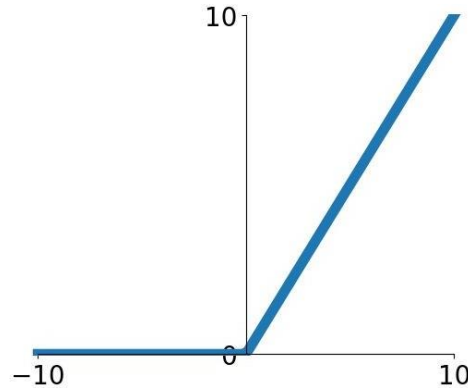
ReLU

(Rectified Linear Unit)

[Krizhevsky et al., 2012]

- Computes $f(x) = \max(0, x)$
- *Does not saturate (in +region)*
- *Very computationally efficient*
- *Converges much faster than sigmoid/tanh in practice (e.g. 6x)*

Activation Functions



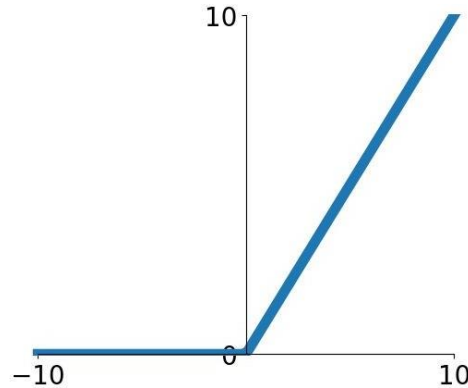
ReLU

(Rectified Linear Unit)

[Krizhevsky et al., 2012]

- Computes $f(x) = \max(0, x)$
- Does not saturate (in +region)
- Very computationally efficient
- Converges much faster than sigmoid/tanh in practice (e.g. 6x)
- Not zero-centered output

Activation Functions



ReLU

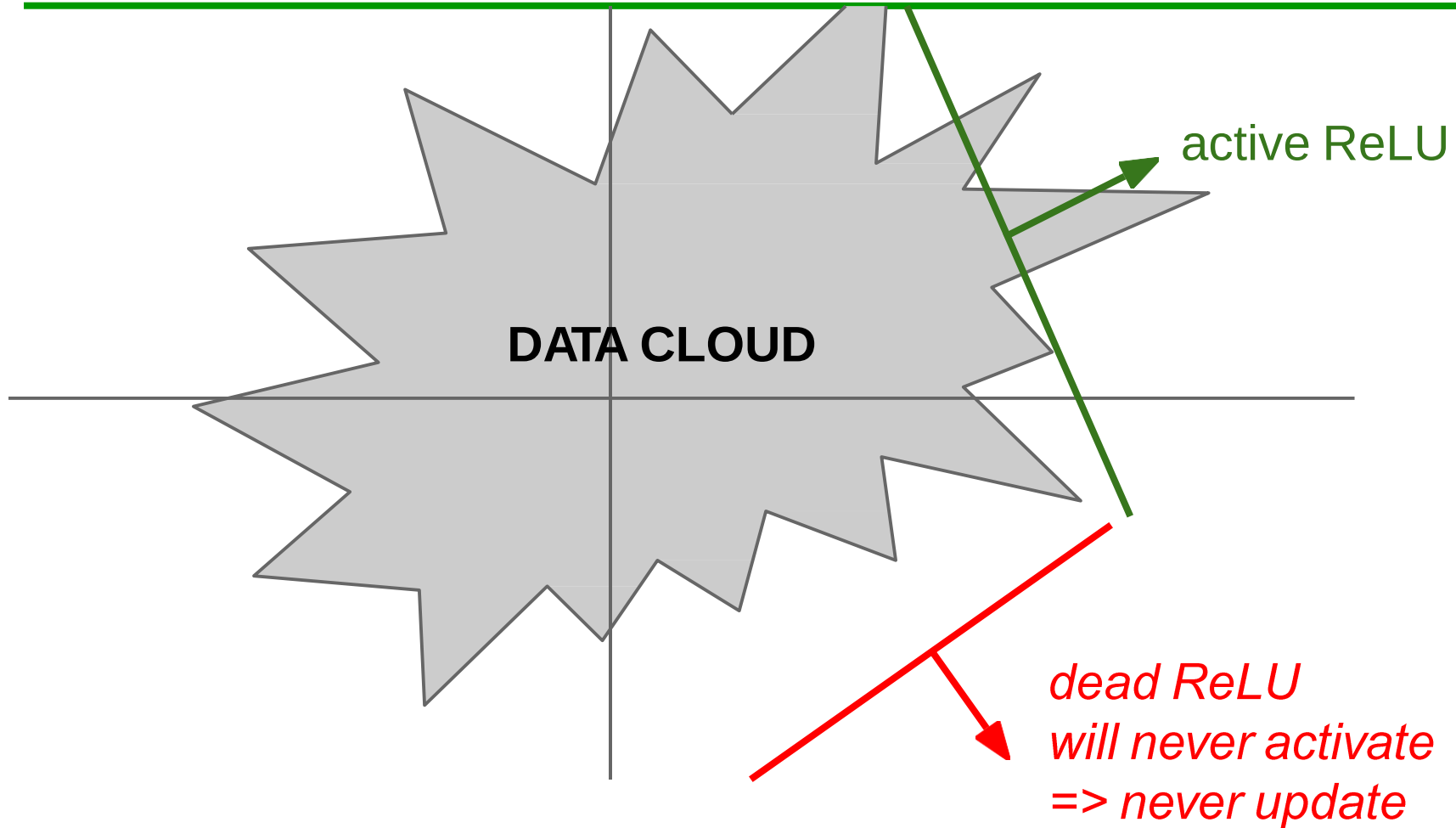
(Rectified Linear Unit)

[Krizhevsky et al., 2012]

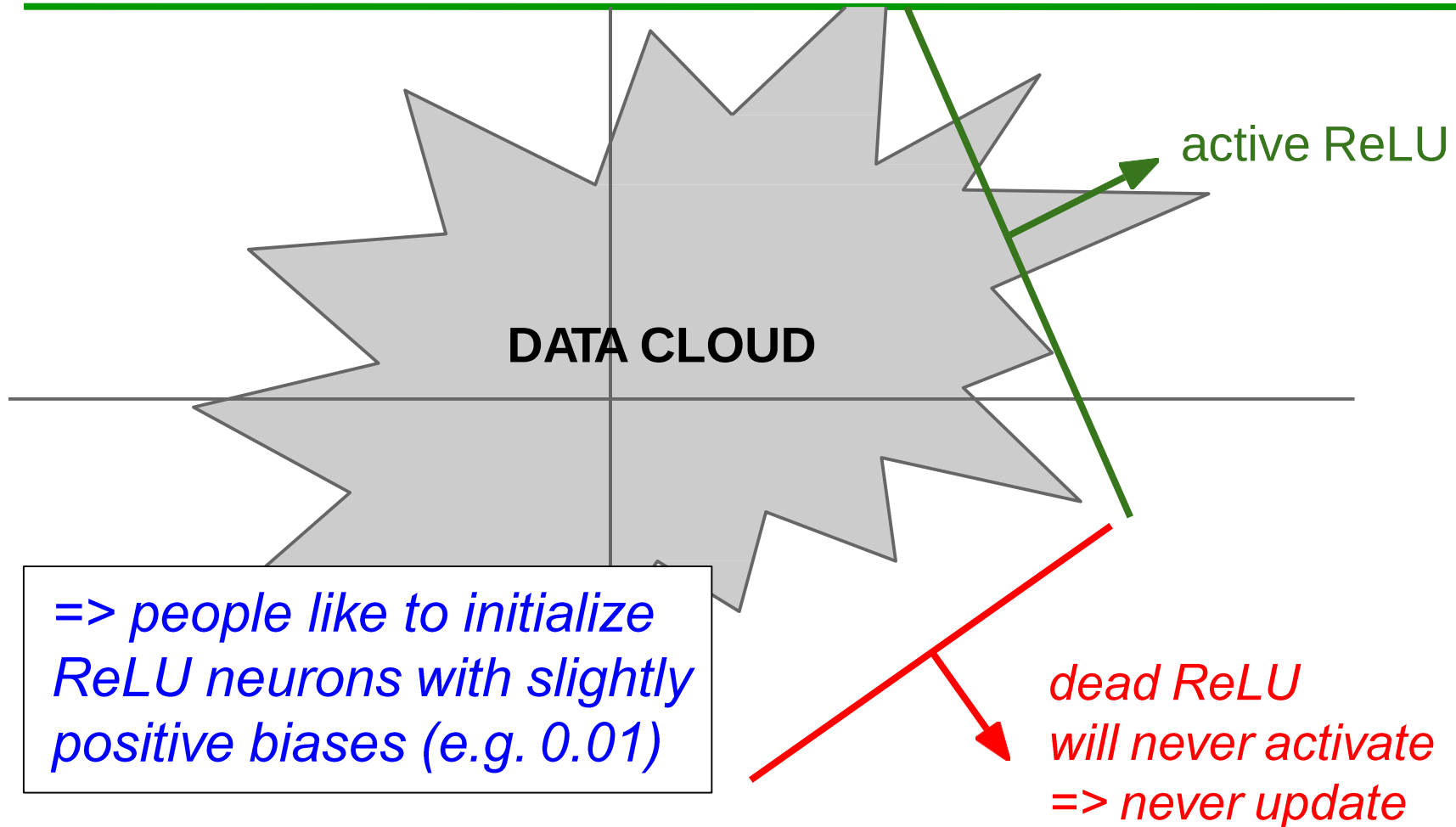
- Computes $f(x) = \max(0, x)$
- Does not saturate (in +region)
- Very computationally efficient
- Converges much faster than sigmoid/tanh in practice (e.g. 6x)
- Not zero-centered output
- An annoyance:

hint: what is the gradient when $x < 0$?

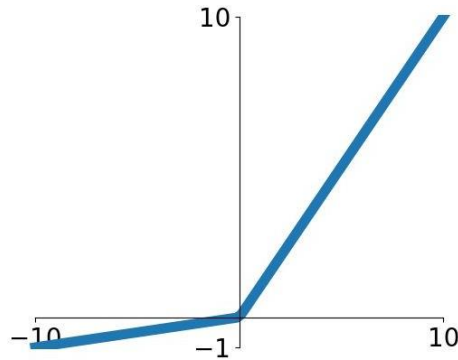
Activation Functions



Activation Functions



Activation Functions



Leaky ReLU

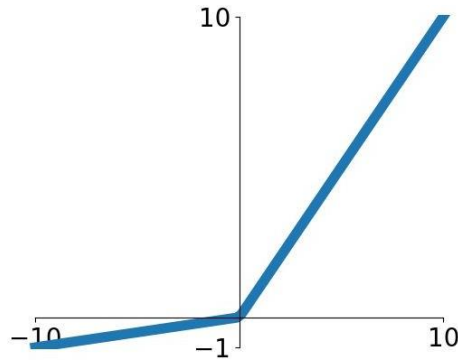
$$f(x) = \max(0.01x, x)$$

[Mass et al., 2013]

[He et al., 2015]

- *Does not saturate*
- *Computationally efficient*
- *Converges much faster than sigmoid/tanh in practice! (e.g. 6x)*
- **will not “die”.**

Activation Functions



Leaky ReLU

$$f(x) = \max(0.01x, x)$$

[Mass et al., 2013]

[He et al., 2015]

- Does not saturate
- Computationally efficient
- Converges much faster than sigmoid/tanh in practice! (e.g. 6x)
- **will not “die”.**

Parametric Rectifier (PReLU)

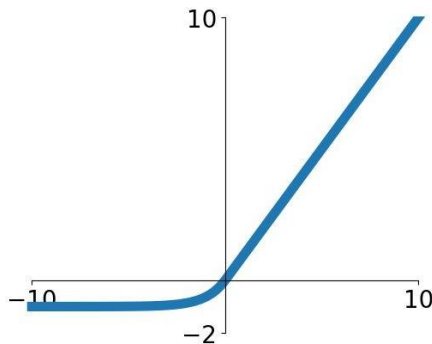
$$f(x) = \max(\alpha x, x)$$

backprop into α (parameter)

Activation Functions

[Clevert et al., 2015]

Exponential Linear Units (ELU)



- *All benefits of ReLU*
- *Closer to zero mean outputs*
- *Negative saturation regime compared with Leaky ReLU adds some robustness to noise*

$$f(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha (\exp(x) - 1) & \text{if } x \leq 0 \end{cases}$$

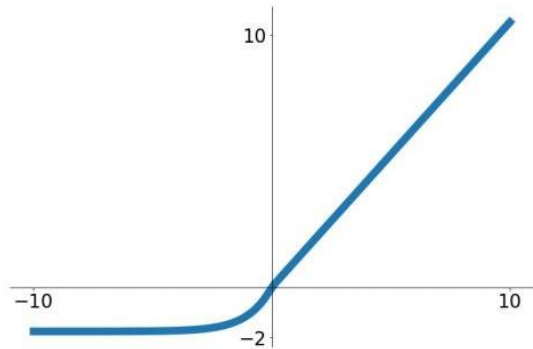
(Alpha default = 1)

- *Computation requires exp()*

Activation Functions

[Klambauer et al. ICLR 2017]

Scaled Exponential Linear Units (SELU)



- Scaled version of ELU that works better for deep networks
- “Self-normalizing” property;
- Can train deep SELU networks without BatchNorm

$$f(x) = \begin{cases} \lambda x & \text{if } x > 0 \\ \lambda \alpha (e^x - 1) & \text{otherwise} \end{cases}$$

$$\alpha = 1.6732632423543772848170429916717$$

$$\lambda = 1.0507009873554804934193349852946$$

Activation Functions

Maxout “Neuron”

[Goodfellow et al., 2013]

- *Does not have the basic form of dot product -> nonlinearity*
- *Generalizes ReLU and Leaky ReLU*
- *Linear Regime! Does not saturate! Does not die!*

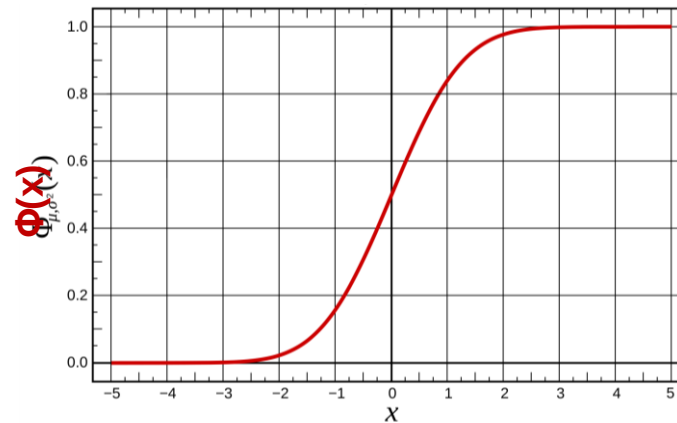
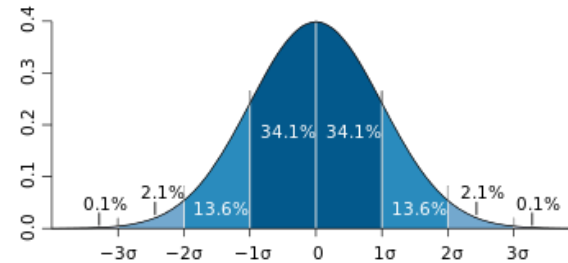
$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

Problem: doubles the number of parameters/neuron :(

Activation Functions

[Hendrycks et al., 2016]

- Computes $f(x) = x * \Phi(x)$



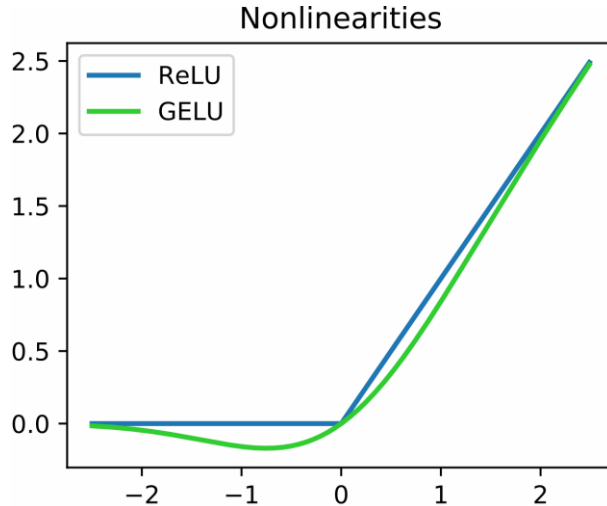
GELU
(Gaussian Error
Linear Unit)

Sources:
https://en.wikipedia.org/wiki/Normal_distribution,
https://en.m.wikipedia.org/wiki/File:Cumulative_distribution_function_for_normal_distribution_mean_0_and_sd_1.png

Activation Functions

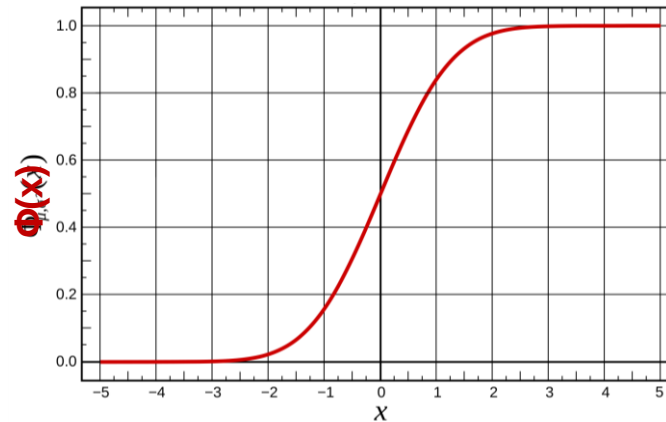
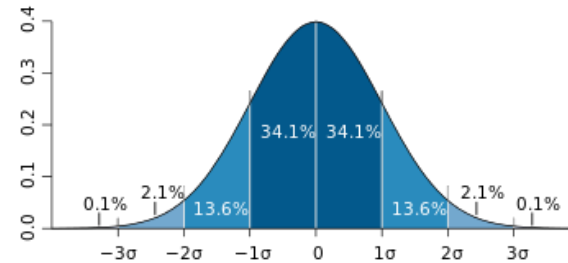
[Hendrycks et al., 2016]

- Computes $f(x) = x * \Phi(x)$



Source: https://en.m.wikipedia.org/wiki/File:ReLU_and_GELU.svg

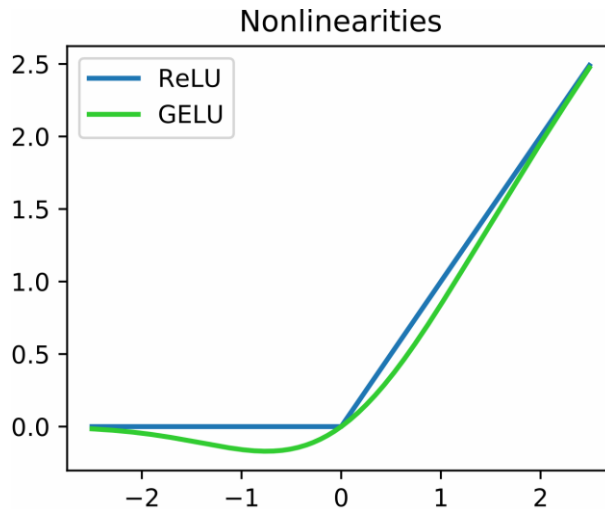
GELU
(Gaussian Error
Linear Unit)



Sources:
https://en.wikipedia.org/wiki/Normal_distribution,
https://en.m.wikipedia.org/wiki/File:Cumulative_distribution_function_for_normal_distribution_mean_0_and_sd_1.png

Activation Functions

[Hendrycks et al., 2016]



Source: https://en.m.wikipedia.org/wiki/File:ReLU_and_GELU.svg

GELU
(Gaussian Error
Linear Unit)

- Computes $\mathbf{f}(\mathbf{x}) = \mathbf{x} * \Phi(\mathbf{x})$
- Very nice behavior around 0
- Smoothness facilitates training in practice
- Higher computational cost than ReLU
- Large negative values can still have gradient $\rightarrow 0$

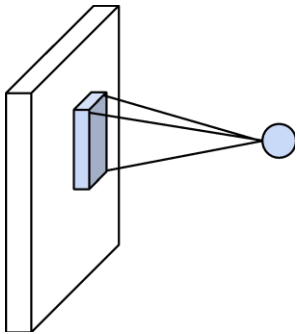
Activation Functions

TLDR: In practice:

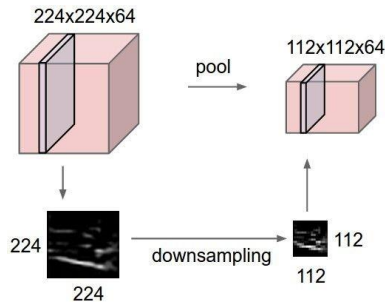
- Use *ReLU*. Be careful with your learning rates
- Try out *Leaky ReLU* / *PReLU* / *Maxout* / *ELU* / *SELU* / *GELU*
 - To squeeze out some marginal gains
- Don't use *sigmoid* or *tanh*

Components of CNNs

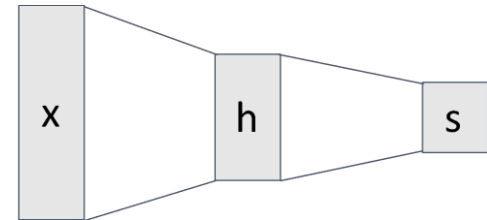
Convolution Layers



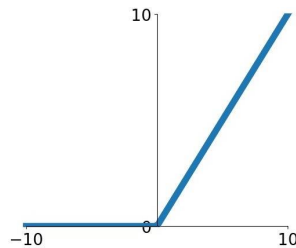
Pooling Layers



Fully-Connected Layers



Activation Function



Normalization

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}$$

Batch Normalization

Consider a single layer $y = Wx$

The following could lead to tough optimization:

- Inputs x are not centered around zero (need large bias)*
- Inputs x have different scaling per-element (entries in W will need to vary a lot)*

Idea: force inputs to be "nicely scaled" at each layer!

Batch Normalization

[Ioffe and Szegedy, 2015]

“you want zero-mean unit-variance activations? just make them so.”

consider a batch of activations at some layer. To make each dimension zero-mean unit-variance, apply:

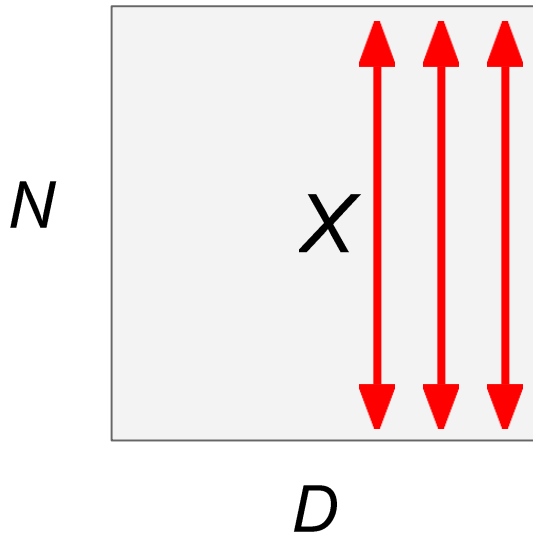
$$\hat{x}^{(k)} = \frac{x^{(k)} - \mathbb{E}[x^{(k)}]}{\sqrt{\text{Var}[x^{(k)}]}}$$

*this is a vanilla
differentiable function...*

Batch Normalization

[Ioffe and Szegedy, 2015]

Input: $x : N \times D$



$$\mu_j = \frac{1}{N} \sum_{i=1}^N x_{i,j} \quad \text{Per-channel mean, shape is } D$$

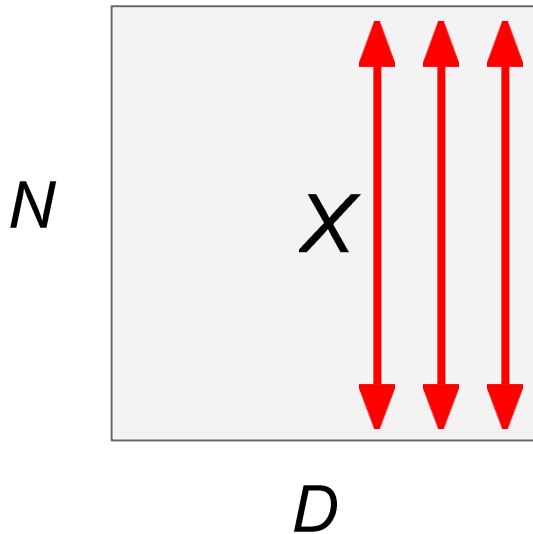
$$\sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (x_{i,j} - \mu_j)^2 \quad \text{Per-channel var, shape is } D$$

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}} \quad \text{Normalized } x, \text{ Shape is } N \times D$$

Batch Normalization

[Ioffe and Szegedy, 2015]

Input: $x : N \times D$



$$\mu_j = \frac{1}{N} \sum_{i=1}^N x_{i,j} \quad \text{Per-channel mean, shape is } D$$

$$\sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (x_{i,j} - \mu_j)^2 \quad \text{Per-channel var, shape is } D$$

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}} \quad \text{Normalized } x, \text{ Shape is } N \times D$$

Problem: What if zero-mean, unit variance is too hard of a constraint?

Batch Normalization

[Ioffe and Szegedy, 2015]

Input: $x : N \times D$

Learnable scale and shift parameters:

$$\gamma, \beta : D$$

*Learning $\gamma = \sigma$,
 $\beta = \mu$ will recover the
identity function!*

$$\mu_j = \frac{1}{N} \sum_{i=1}^N x_{i,j} \quad \text{Per-channel mean, shape is } D$$

$$\sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (x_{i,j} - \mu_j)^2 \quad \text{Per-channel var, shape is } D$$

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}} \quad \text{Normalized } x, \text{ Shape is } N \times D$$

$$y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j$$

Batch Normalization: Test-Time

*Estimates depend on minibatch;
can't do this at test-time!*

Input: $x : N \times D$

**Learnable scale and
shift parameters:**

$$\gamma, \beta : D$$

*Learning $\gamma = \sigma$,
 $\beta = \mu$ will recover the
identity function!*

$$\mu_j = \frac{1}{N} \sum_{i=1}^N x_{i,j} \quad \text{Per-channel mean, shape is } D$$
$$\sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (x_{i,j} - \mu_j)^2 \quad \text{Per-channel var, shape is } D$$

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}} \quad \text{Normalized } x, \text{ Shape is } N \times D$$

$$y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j$$

Batch Normalization: Test-Time

Input: $x : N \times D$

$\mu_j =$ (Running) average of values seen during training
Per-channel mean, shape is D

Learnable scale and shift parameters:

$\gamma, \beta : D$

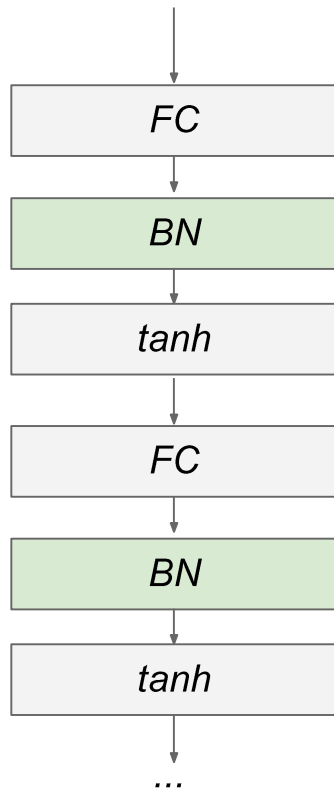
$\sigma_j^2 =$ (Running) average of values seen during training
Per-channel var, shape is D

*During testing batchnorm becomes a linear operator!
Can be fused with the previous fully-connected or conv layer*

$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}$
Normalized x ,
Shape is $N \times D$

$y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j$
Output,
Shape is $N \times D$

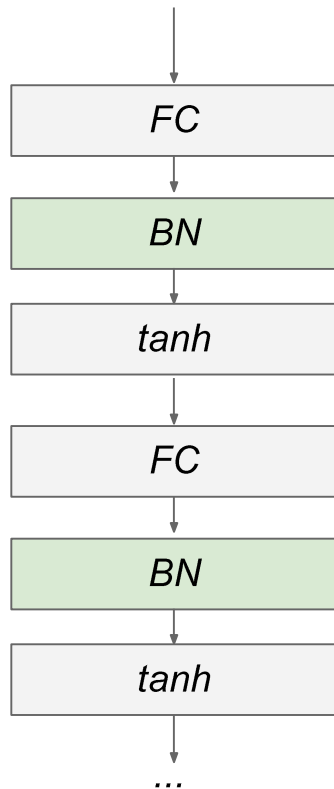
Batch Normalization



Usually inserted after Fully Connected or Convolutional layers, and before nonlinearity.

$$\hat{x}^{(k)} = \frac{x^{(k)} - \mathbb{E}[x^{(k)}]}{\sqrt{\text{Var}[x^{(k)}]}}$$

Batch Normalization



- Makes deep networks **much** easier to train!
- Improves gradient flow
- Allows higher learning rates, faster convergence
- Networks become more robust to initialization
- Acts as regularization during training
- Nearly zero overhead at test-time: can be fused with conv!
- Behaves differently during training and testing: this is a very common source of bugs!

Batch Normalization for ConvNets

*Batch Normalization for
fully-connected networks*

$$\mathbf{x}: \mathbf{N} \times \mathbf{D}$$

Normalize



$$\boldsymbol{\mu}, \boldsymbol{\sigma}: 1 \times \mathbf{D}$$

$$\boldsymbol{\gamma}, \boldsymbol{\beta}: 1 \times \mathbf{D}$$

$$\mathbf{y} = \boldsymbol{\gamma} (\mathbf{x} - \boldsymbol{\mu}) / \boldsymbol{\sigma} + \boldsymbol{\beta}$$

*Batch Normalization for
convolutional networks
(Spatial Batchnorm, BatchNorm2D)*

$$\mathbf{x}: \mathbf{N} \times \mathbf{C} \times \mathbf{H} \times \mathbf{W}$$

Normalize



$$\boldsymbol{\mu}, \boldsymbol{\sigma}: 1 \times \mathbf{C} \times 1 \times 1$$

$$\boldsymbol{\gamma}, \boldsymbol{\beta}: 1 \times \mathbf{C} \times 1 \times 1$$

$$\mathbf{y} = \boldsymbol{\gamma} (\mathbf{x} - \boldsymbol{\mu}) / \boldsymbol{\sigma} + \boldsymbol{\beta}$$

Layer Normalization

Batch Normalization for
fully-connected networks

$$\begin{array}{l} \mathbf{x}: \mathbf{N} \times \mathbf{D} \\ \text{Normalize} \downarrow \\ \boxed{\begin{array}{l} \boldsymbol{\mu}, \boldsymbol{\sigma}: 1 \times \mathbf{D} \\ \boldsymbol{\gamma}, \boldsymbol{\beta}: 1 \times \mathbf{D} \end{array}} \\ \mathbf{y} = \boldsymbol{\gamma}(\mathbf{x} - \boldsymbol{\mu}) / \boldsymbol{\sigma} + \boldsymbol{\beta} \end{array}$$

Layer Normalization for
fully-connected networks
Same behavior at train and test!
Can be used in recurrent networks
(dynamically-changing inputs)

$$\begin{array}{l} \mathbf{x}: \mathbf{N} \times \mathbf{D} \\ \text{Normalize} \downarrow \\ \boxed{\begin{array}{l} \boldsymbol{\mu}, \boldsymbol{\sigma}: \mathbf{N} \times 1 \\ \boldsymbol{\gamma}, \boldsymbol{\beta}: 1 \times \mathbf{D} \end{array}} \\ \mathbf{y} = \boldsymbol{\gamma}(\mathbf{x} - \boldsymbol{\mu}) / \boldsymbol{\sigma} + \boldsymbol{\beta} \end{array}$$

Ba, Kiros, and Hinton, "Layer Normalization", arXiv 2016

Instance Normalization

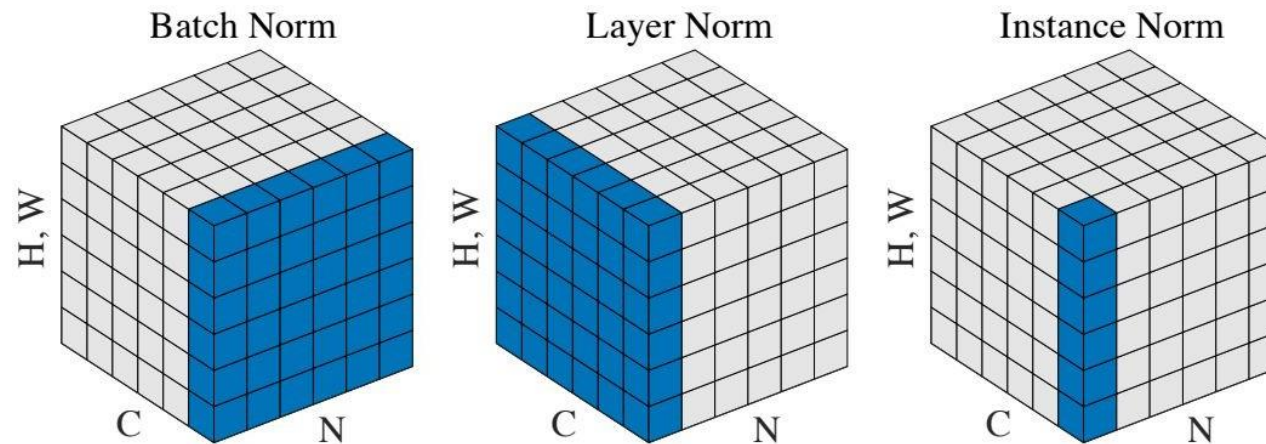
Batch Normalization for
convolutional networks

Instance Normalization for
convolutional networks
Same behavior at train / test!

$$\begin{array}{l} \mathbf{x} : N \times C \times H \times W \\ \text{Normalize} \quad \downarrow \quad \downarrow \quad \downarrow \\ \boldsymbol{\mu}, \boldsymbol{\sigma} : 1 \times C \times 1 \times 1 \\ \boldsymbol{\gamma}, \boldsymbol{\beta} : 1 \times C \times 1 \times 1 \\ \mathbf{y} = \boldsymbol{\gamma} (\mathbf{x} - \boldsymbol{\mu}) / \boldsymbol{\sigma} + \boldsymbol{\beta} \end{array}$$

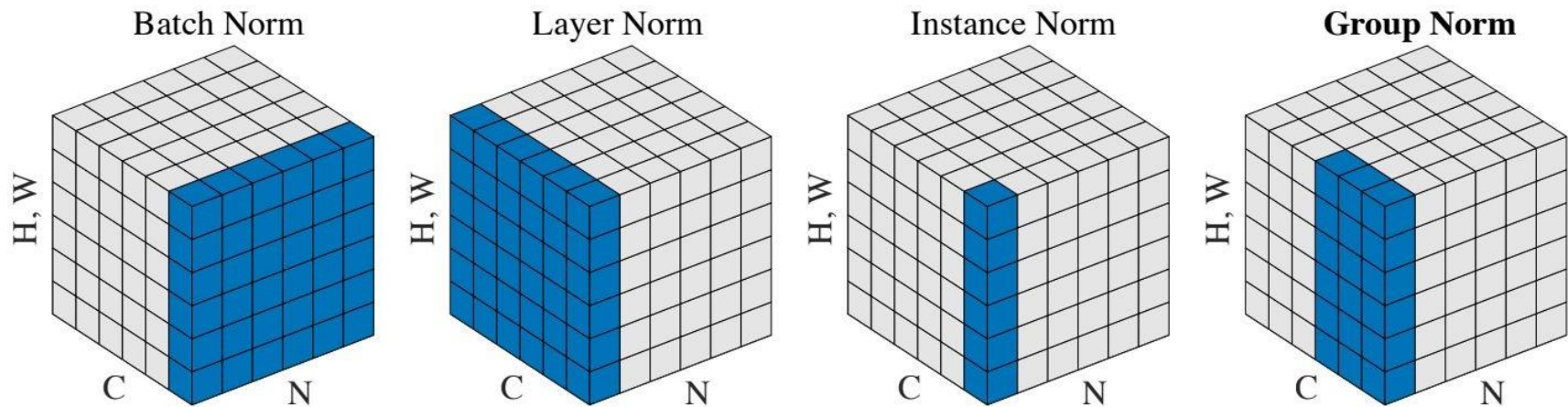
$$\begin{array}{l} \mathbf{x} : N \times C \times H \times W \\ \text{Normalize} \quad \quad \downarrow \quad \downarrow \\ \boldsymbol{\mu}, \boldsymbol{\sigma} : N \times C \times 1 \times 1 \\ \boldsymbol{\gamma}, \boldsymbol{\beta} : 1 \times C \times 1 \times 1 \\ \mathbf{y} = \boldsymbol{\gamma} (\mathbf{x} - \boldsymbol{\mu}) / \boldsymbol{\sigma} + \boldsymbol{\beta} \end{array}$$

Comparison of Normalization Layers



Wu and He, "Group Normalization", ECCV 2018

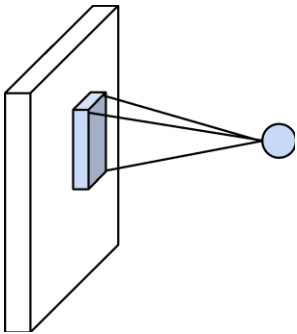
Group Normalization



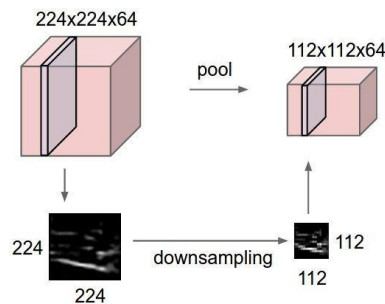
Wu and He, "Group Normalization", ECCV 2018

Components of CNNs

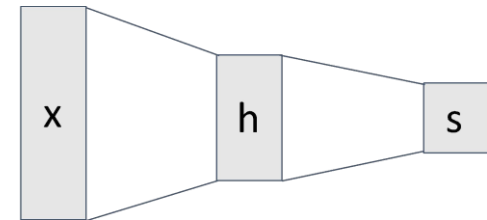
Convolution Layers



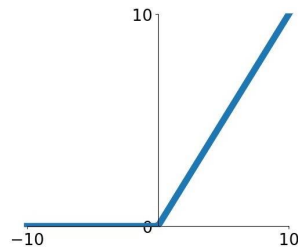
Pooling Layers



Fully-Connected Layers



Activation Function



Normalization

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}$$

Question: How should we put them together?

